



RESEARCH CONNECT

Algorithm-Based Research Paper Discovery and Analysis System

PROJECT REPORT

Submitted in partial fulfilment of the requirements for the course

Data Structures and Algorithms – 3 (25CS2103E)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Submitted by

K.V. KOUSHAL — 2520030009

Y. CHARAN SAI — 2520030173

K. SHASHANK — 2520030215

Under the guidance of

Dr. V. SIREESHA, Professor

Department of Computer Science and Engineering

Academic Year 2026–2027

DECLARATION

We hereby declare that the project report entitled “**Research Connect**” is a record of work carried out by K.V. Koushal (2520030009), Y. Charan Sai (2520030173), and K. Shashank (2520030215), in partial fulfilment of the requirements for the course Data Structures and Algorithms – 3 in the Department of Computer Science and Engineering. The report describes the design, implementation approach, algorithms, and evaluation of a research-paper discovery and analysis application.

We affirm that the explanations and project-specific descriptions presented here are prepared for academic submission and are based on our project materials. Wherever external technical documentation or scholarly resources informed the work, those resources are identified in the references. Any results described as system capabilities refer to the functions implemented or intended in the supplied project; they should not be interpreted as independently benchmarked performance claims unless a measured value is explicitly stated.

We understand that academic integrity requires proper attribution, honest reporting, and clear distinction between implemented features and proposed future improvements. We accept responsibility for reviewing the report, verifying the accuracy of the final submission, and making corrections requested by the course instructor.

K.V. Koushal — 2520030009 Y. Charan Sai — 2520030173 K. Shashank — 2520030215

CERTIFICATE

This is to certify that the project report entitled **“Research Connect”** is submitted by K.V. Koushal, Y. Charan Sai, and K. Shashank of the Department of Computer Science and Engineering, in partial fulfilment of the requirements for the Bachelor of Technology programme during the academic year 2026–2027.

The project applies Data Structures and Algorithms to the practical problem of discovering, searching, comparing, and exploring research papers. The work brings together string-matching methods, indexing structures, text-similarity techniques, and graph traversal in a full-stack application architecture. The report is presented for academic review and assessment.

The signatures below may be completed by the appropriate faculty members after reviewing the project and report.

Project Guide: Dr. V. Sireesha, Professor Signature: _____

Head of Department: Dr. P. Venkateshwara Rao Signature: _____

External Examiner: _____ Signature: _____

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, Dr. V. Sireesha, Professor, Department of Computer Science and Engineering, for her guidance, support, and constructive suggestions throughout the development of Research Connect. Her advice helped us understand the project requirements and connect theoretical algorithm concepts with a practical software system.

We thank Dr. P. Venkateshwara Rao, Associate Professor and Head of the Department of Computer Science and Engineering, for providing encouragement and an academic environment in which we could develop and document the project. We are also grateful to the faculty members who taught us programming, data structures, algorithms, database concepts, and web application development. These subjects provided the foundation for the system described in this report.

Finally, we thank our classmates, friends, and everyone who offered feedback or assistance during planning, coding, testing, and report preparation. The project has been a valuable learning experience in teamwork, problem decomposition, software integration, debugging, and technical communication. We acknowledge that continued testing and improvement will be important as the corpus and feature set grow.

K.V. Koushal • Y. Charan Sai • K. Shashank

ABSTRACT

Research Connect is a research-paper discovery and analysis application intended to help students, academicians, and researchers locate relevant technical literature in an organised manner. As the volume of scientific publications increases, a user may spend considerable time searching different sources, correcting query spelling, comparing documents, and following citation relationships. Research Connect brings several discovery functions into one interface and demonstrates how Data Structures and Algorithms can support information retrieval.

The project materials describe a controlled corpus of 180 research-paper records: 15 foundational papers and 165 synthetic records across domains including artificial intelligence, natural language processing, cybersecurity, quantum computing, robotics, and cloud computing. Search can use terms and paper metadata such as title, author, year, and category. KMP supports exact pattern matching; Aho–Corasick supports simultaneous matching of multiple patterns; Levenshtein edit distance supports spelling-tolerant suggestions; suffix arrays and longest-common-prefix information support textual-pattern comparison; and graph traversal explores citation links.

The proposed application uses a React.js and Vite frontend with a Java Spring Boot backend communicating through REST APIs. Hash-based lookup and an inverted index help narrow candidate records, while a trie can support prefix suggestions. This report explains the problem, requirements, design, data structures, algorithm choices, modules, testing approach, limitations, and future development opportunities. The project is primarily an educational demonstration; its usefulness and speed at larger scale depend on corpus quality, implementation details, and systematic performance evaluation.

TABLE OF CONTENTS

Preliminary pages: Cover page; Declaration; Certificate; Acknowledgement; Abstract; Table of Contents; List of Figures and Tables.

Chapter 1 — Introduction: Background, problem statement, motivation, scope, objectives, users, and use cases.

Chapter 2 — System Requirements: Functional and non-functional requirements, hardware and software requirements, assumptions, constraints, and feasibility.

Chapter 3 — System Design: Architecture, layers, data flow, data model, data structures, API design, and security considerations.

Chapter 4 — Algorithms and Implementation: Corpus loading, KMP, Aho–Corasick, edit distance, suffix array and LCP, citation graph, indexing, frontend, backend, and API integration.

Chapter 5 — Testing and Results: Test strategy, functional test cases, algorithm examples, result interpretation, performance considerations, and evaluation criteria.

Chapter 6 — Challenges and Limitations: Development challenges, solutions, current limitations, and risks.

Chapter 7 — Conclusion and Future Work: Summary, future enhancements, lessons learned, and references.

Page numbers are assigned automatically in the PDF. Each major chapter begins on a fresh page, and the technical sections are expanded to use the available page area more effectively.

CHAPTER 1 — INTRODUCTION

1.1 Background and Overview

Research papers communicate new methods, experimental results, surveys, and open problems. Students often begin a project by searching for a topic, reading abstracts, comparing methods, and following references to earlier work. The number of publications makes this process difficult when information is spread across different websites or when a local collection contains many documents. A simple search box is useful, but it may not handle spelling errors, multiple search terms, similarity between papers, or the relationships created by citations.

Research Connect is designed as a single environment for exploring a controlled collection of papers. It stores descriptive information such as title, author, publication year, research category, text content, and citation relationships. The user can search the collection, narrow the result set, inspect a paper, look for related records, and explore connections between papers. The project is valuable as a learning exercise because each feature can be linked to a particular data structure or algorithm rather than being treated as a black box.

The project description identifies a corpus of 180 records, made up of 15 foundational papers and 165 synthetic records. The distinction matters: synthetic records can support interface and algorithm demonstrations, but they should not be treated as authentic scholarly publications. A production service would need reliable metadata, verified full text, stable identifiers, and careful source attribution. The current scope is therefore a controlled prototype for demonstrating research discovery concepts.

1.2 Problem Statement

The central problem is that locating relevant literature is more complicated than finding an exact word. A user may not know the best vocabulary for a topic, may mistype a technical term, or may want to search several concepts at once. A collection can also contain many papers with overlapping terminology. If the application repeatedly scans every document for every query, work can increase as the corpus grows. Manual comparison is similarly inefficient when users need to identify recurring phrases or trace citation paths.

Research Connect addresses these difficulties by combining indexing, string matching, spelling-tolerant matching, textual-pattern comparison, and graph traversal. KMP provides an efficient way to locate a pattern in text. Aho–Corasick searches for several terms in one scan. Levenshtein edit distance estimates how different two words are. Suffix-array and LCP techniques reveal common text fragments, while BFS and DFS explore citation connections. The inverted index narrows the set of candidate papers before more detailed processing.

The problem is not solved merely by adding algorithms. Results must be understandable, metadata must be consistent, and the interface must communicate when a match is exact, approximate, or based on textual overlap. The application must also distinguish content similarity from scientific relevance. A shared phrase can occur in unrelated papers, and a highly relevant paper may use different terminology. These considerations guide the requirements and limitations discussed later.

These requirements should be translated into observable behaviour. A reviewer should be able to provide an input, follow the user interface, and confirm the expected output without needing to infer what the system intended to do. Requirements that cannot be tested should be clarified or rewritten. This also helps the team decide which features are essential for the current submission and which belong in future work.

The project should be judged against its stated scope rather than against features found in commercial research platforms. A controlled academic prototype can demonstrate sound algorithm selection and modular design even when it does not provide live indexing or a very large corpus. Clear boundaries make the report more credible and help future development focus on improvements that solve a defined problem.

The system should also be assessed from the perspective of the person using it. A technically correct output is not enough if the user cannot understand why it appeared or what action to take next. Labels, examples, sensible defaults, and clear empty states reduce confusion. Where the system makes an approximate decision, such as suggesting a spelling correction or a related paper, the interface should communicate that uncertainty and let the user inspect the underlying record before relying on it.

Finally, the implementation should remain aligned with the evidence available in the project. Features that are implemented should be supported by a demonstration or test; features that are planned should be described as future work. This distinction is especially important in an academic report, where clear reasoning and honest limitations are stronger than unsupported claims. Keeping the documentation, source code, and demonstration consistent will make the project easier to evaluate and extend.

A final consideration is maintainability over time. As new records and search options are added, the team should keep a short change log, update the test cases, and review whether the current data structures still suit the workload. Small, well-documented changes are easier to verify than large rewrites. The project can therefore grow in stages: establish correct results first, improve the user experience next, and only then optimise measured bottlenecks or introduce advanced discovery techniques.

1.3 Motivation and Need

Students beginning research frequently face three practical obstacles: not knowing which terms to search, having too many results to review, and finding it difficult to discover connected work. Research Connect is motivated by the idea that a useful discovery tool should support a journey from a broad query to a smaller, better-organised group of papers. Search suggestions can help users form a query; typo tolerance can rescue an imperfect query; filters can reduce irrelevant results; and citation traversal can expose related work beyond literal word matching.

The project also provides a concrete way to study algorithms. In a classroom, string matching and graph traversal can seem abstract when presented only as pseudocode. In this system, each technique has a clear responsibility and can be tested with realistic input. Students can observe how a KMP prefix table avoids unnecessary comparisons, how a trie shares prefixes, and how a queue supports breadth-first traversal. The project therefore connects algorithmic analysis with software design and user needs.

A further motivation is modularity. If each algorithm is placed in a separate service or class, developers can test and improve one technique without rewriting the whole application. A well-defined REST API separates interface behaviour from algorithm execution. This structure also makes it easier to replace the controlled corpus with a database or external scholarly metadata source in future versions.

These requirements should be translated into observable behaviour. A reviewer should be able to provide an input, follow the user interface, and confirm the expected output without needing to infer what the system intended to do. Requirements that cannot be tested should be clarified or rewritten. This also helps the team decide which features are essential for the current submission and which belong in future work.

The project should be judged against its stated scope rather than against features found in commercial research platforms. A controlled academic prototype can demonstrate sound algorithm selection and modular design even when it does not provide live indexing or a very large corpus. Clear boundaries make the report more credible and help future development focus on improvements that solve a defined problem.

The system should also be assessed from the perspective of the person using it. A technically correct output is not enough if the user cannot understand why it appeared or what action to take next. Labels, examples, sensible defaults, and clear empty states reduce confusion. Where the system makes an approximate decision, such as suggesting a spelling correction or a related paper, the interface should communicate that uncertainty and let the user inspect the underlying record before relying on it.

Finally, the implementation should remain aligned with the evidence available in the project. Features that are implemented should be supported by a demonstration or test; features that are planned should be described as future work. This distinction is especially important in an academic report, where clear reasoning and honest limitations are stronger than unsupported claims. Keeping the documentation, source code, and demonstration consistent will make the project easier to evaluate and extend.

A final consideration is maintainability over time. As new records and search options are added, the team should keep a short change log, update the test cases, and review whether the current data structures still suit the workload. Small, well-documented changes are easier to verify than large rewrites. The project can therefore grow in stages: establish correct results first, improve the user experience next, and only then optimise measured bottlenecks or introduce advanced discovery techniques.

1.4 Scope of the Project

The current scope covers paper loading, keyword search, multiple-keyword search, typo-tolerant suggestions, prefix-based suggestions, textual similarity, citation exploration, year and category filtering, paper details, and collection statistics. The system is intended to make these functions available through an interactive web interface. The backend processes requests and returns structured results that the frontend can display consistently.

The scope is limited to the records and relationships supplied with the project. Search quality depends on how the corpus is prepared, how text is normalised, and how metadata is represented. The system does not automatically guarantee access to every published paper, verify the scientific quality of a publication, or determine whether a paper is trustworthy. It is also not a replacement for reading a paper carefully or consulting a domain expert.

Features such as semantic embeddings, personalised recommendation, live indexing of scholarly websites, user accounts, saved libraries, advanced access control, and large-scale distributed storage are potential extensions rather than assumptions about the current implementation. Clearly defining the scope prevents the report from overstating what the prototype demonstrates and gives future developers a practical roadmap.

These requirements should be translated into observable behaviour. A reviewer should be able to provide an input, follow the user interface, and confirm the expected output without needing to infer what the system intended to do. Requirements that cannot be tested should be clarified or rewritten. This also helps the team decide which features are essential for the current submission and which belong in future work.

The project should be judged against its stated scope rather than against features found in commercial research platforms. A controlled academic prototype can demonstrate sound algorithm selection and modular design even when it does not provide live indexing or a very large corpus. Clear boundaries make the report more credible and help future development focus on improvements that solve a defined problem.

The system should also be assessed from the perspective of the person using it. A technically correct output is not enough if the user cannot understand why it appeared or what action to take next. Labels, examples, sensible defaults, and clear empty states reduce confusion. Where the system makes an approximate decision, such as suggesting a spelling correction or a related paper, the interface should communicate that uncertainty and let the user inspect the underlying record before relying on it.

Finally, the implementation should remain aligned with the evidence available in the project. Features that are implemented should be supported by a demonstration or test; features that are planned should be described as future work. This distinction is especially important in an academic report, where clear reasoning and honest limitations are stronger than unsupported claims. Keeping the documentation, source code, and demonstration consistent will make the project easier to evaluate and extend.

A final consideration is maintainability over time. As new records and search options are added, the team should keep a short change log, update the test cases, and review whether the current data structures still suit the workload. Small, well-documented changes are easier to verify than large rewrites. The project can therefore grow in stages: establish correct results first, improve the user experience next, and only then optimise measured bottlenecks or introduce advanced discovery techniques.

1.5 Project Objectives

The first objective is to make research-paper retrieval organised and efficient. Users should be able to search with keywords and metadata, inspect matching records, and narrow results using filters. The second objective is to demonstrate algorithmic variety: exact pattern matching, simultaneous pattern matching, edit-distance calculation, suffix-based comparison, graph traversal, hashing, and inverted indexing. Each algorithm is selected because it addresses a distinct subproblem in discovery.

The third objective is to integrate the algorithms into a full-stack application. React components should handle input, state, result presentation, and user navigation. Spring Boot controllers should accept requests, validate parameters, call service methods, and return consistent responses. The fourth objective is maintainability: algorithms should have clear interfaces, independent tests, and documented complexity. The fifth objective is to support learning by explaining how the underlying data structures contribute to the visible behaviour.

Success should be assessed through testable criteria rather than general claims. Example criteria include whether exact terms are found in known text, whether multiple terms can be reported in one scan, whether typo suggestions are plausible for controlled examples, whether filters exclude records outside the chosen criteria, and whether graph traversal visits reachable nodes without repeating them. Performance should be measured with defined corpus sizes and repeatable experiments before making numerical speed claims.

These requirements should be translated into observable behaviour. A reviewer should be able to provide an input, follow the user interface, and confirm the expected output without needing to infer what the system intended to do. Requirements that cannot be tested should be clarified or rewritten. This also helps the team decide which features are essential for the current submission and which belong in future work.

The project should be judged against its stated scope rather than against features found in commercial research platforms. A controlled academic prototype can demonstrate sound algorithm selection and modular design even when it does not provide live indexing or a very large corpus. Clear boundaries make the report more credible and help future development focus on improvements that solve a defined problem.

The system should also be assessed from the perspective of the person using it. A technically correct output is not enough if the user cannot understand why it appeared or what action to take next. Labels, examples, sensible defaults, and clear empty states reduce confusion. Where the system makes an approximate decision, such as suggesting a spelling correction or a related paper, the interface should communicate that uncertainty and let the user inspect the underlying record before relying on it.

Finally, the implementation should remain aligned with the evidence available in the project. Features that are implemented should be supported by a demonstration or test; features that are planned should be described as future work. This distinction is especially important in an academic report, where clear reasoning and honest limitations are stronger than unsupported claims. Keeping the documentation, source code, and demonstration consistent will make the project easier to evaluate and extend.

A final consideration is maintainability over time. As new records and search options are added, the team should keep a short change log, update the test cases, and review whether the current data structures still suit the workload. Small, well-documented changes are easier to verify than large rewrites. The project can therefore grow in stages: establish correct results first, improve the user experience next, and only then optimise measured bottlenecks or introduce advanced discovery techniques.

CHAPTER 2 — SYSTEM REQUIREMENTS

2.1 Functional Requirements

Functional requirements describe what the application must do. Paper search allows a user to search by keyword or paper information. Filtering allows results to be narrowed by publication year and research category. Exact search locates occurrences of a pattern with KMP. Multiple-keyword search uses Aho–Corasick to search a collection of patterns in one text traversal. Typo-tolerant search uses edit distance to find vocabulary entries that may be intended by the user.

The similarity function identifies papers with common textual patterns and offers them as possible related records. Citation analysis models paper relationships as a directed graph. BFS and DFS provide different ways to traverse that graph. A statistics function reports summary information such as record counts, categories, years, authors, and citation relationships. A paper-detail function presents available metadata and content for one selected record.

Each functional requirement should have a clear input, processing rule, and expected output. For example, a search request includes a query and optional filters; the backend returns a list of paper records and may include match details. Invalid parameters should produce an understandable error instead of an application crash. These behaviours can be tested independently through service-level tests and end-to-end interface checks.

These requirements should be translated into observable behaviour. A reviewer should be able to provide an input, follow the user interface, and confirm the expected output without needing to infer what the system intended to do. Requirements that cannot be tested should be clarified or rewritten. This also helps the team decide which features are essential for the current submission and which belong in future work.

The project should be judged against its stated scope rather than against features found in commercial research platforms. A controlled academic prototype can demonstrate sound algorithm selection and modular design even when it does not provide live indexing or a very large corpus. Clear boundaries make the report more credible and help future development focus on improvements that solve a defined problem.

The system should also be assessed from the perspective of the person using it. A technically correct output is not enough if the user cannot understand why it appeared or what action to take next. Labels, examples, sensible defaults, and clear empty states reduce confusion. Where the system makes an approximate decision, such as suggesting a spelling correction or a related paper, the interface should communicate that uncertainty and let the user inspect the underlying record before relying on it.

Finally, the implementation should remain aligned with the evidence available in the project. Features that are implemented should be supported by a demonstration or test; features that are planned should be described as future work. This distinction is especially important in an academic report, where clear reasoning and honest limitations are stronger than unsupported claims. Keeping the documentation, source code, and demonstration consistent will make the project easier to evaluate and extend.

A final consideration is maintainability over time. As new records and search options are added, the team should keep a short change log, update the test cases, and review whether the current data structures still suit the workload. Small, well-documented changes are easier to verify than large rewrites. The project can therefore grow in stages: establish correct results first, improve the user experience next, and only then optimise measured bottlenecks or introduce advanced discovery techniques.

2.2 Non-Functional Requirements

Performance concerns the time and memory needed to process a query. Indexing can reduce the number of documents that must be inspected, but index construction itself has a cost and indexes consume memory. Usability means that search controls, filters, result cards, and paper details are understandable without requiring users to know the algorithms. Reliability means that the same corpus and request should produce consistent results, apart from explicitly random or personalised features.

Maintainability requires modular code, meaningful names, limited duplication, documented API contracts, and tests. Scalability means that the architecture can accommodate more records without a complete redesign, although actual scale depends on storage, indexing, concurrency, and deployment resources. Compatibility concerns the supported browser, Java runtime, Node.js tooling, and API configuration. Accessibility considerations include readable contrast, keyboard navigation, descriptive labels, and helpful error messages.

Security and privacy are important even for an academic prototype. User input must be validated; errors should not expose stack traces; cross-origin access should be restricted to intended frontend origins in a deployed system; and external documents should be handled carefully. If accounts or personal libraries are added, authentication, authorisation, secure storage, and privacy policies become necessary. Non-functional requirements should be evaluated explicitly rather than assumed to be satisfied because the application runs locally.

These requirements should be translated into observable behaviour. A reviewer should be able to provide an input, follow the user interface, and confirm the expected output without needing to infer what the system intended to do. Requirements that cannot be tested should be clarified or rewritten. This also helps the team decide which features are essential for the current submission and which belong in future work.

The project should be judged against its stated scope rather than against features found in commercial research platforms. A controlled academic prototype can demonstrate sound algorithm selection and modular design even when it does not provide live indexing or a very large corpus. Clear boundaries make the report more credible and help future development focus on improvements that solve a defined problem.

The system should also be assessed from the perspective of the person using it. A technically correct output is not enough if the user cannot understand why it appeared or what action to take next. Labels, examples, sensible defaults, and clear empty states reduce confusion. Where the system makes an approximate decision, such as suggesting a spelling correction or a related paper, the interface should communicate that uncertainty and let the user inspect the underlying record before relying on it.

Finally, the implementation should remain aligned with the evidence available in the project. Features that are implemented should be supported by a demonstration or test; features that are planned should be described as future work. This distinction is especially important in an academic report, where clear reasoning and honest limitations are stronger than unsupported claims. Keeping the documentation, source code, and demonstration consistent will make the project easier to evaluate and extend.

A final consideration is maintainability over time. As new records and search options are added, the team should keep a short change log, update the test cases, and review whether the current data structures still suit the workload. Small, well-documented changes are easier to verify than large rewrites. The project can therefore grow in stages: establish correct results first, improve the user experience next, and only then optimise measured bottlenecks or introduce advanced discovery techniques.

2.3 Hardware Requirements

The supplied report identifies a standard laptop or desktop with an Intel Core i3 processor or equivalent, at least 4 GB of RAM, and approximately 500 MB of free storage as a minimum development environment. These values are practical baseline estimates for a small controlled corpus, not universal limits. A computer with more memory may improve the experience when an IDE, frontend development server, backend server, browser, and test tools are running together.

Processor requirements depend on the number of records, length of text, number of queries, and complexity of the similarity operations. Index construction and suffix-array operations may require more time and memory than simple metadata filtering. Disk needs include the source code, dependency caches, corpus files, logs, generated build files, and report materials. Keeping adequate free storage prevents builds and package installations from failing unexpectedly.

For repeatable performance testing, record the hardware configuration and software versions. Compare timings on the same machine and use the same dataset and query set. A result measured on one laptop should not be presented as a universal speed claim. If the project is deployed to a server, the server specifications and expected number of concurrent users should be documented separately from the student development environment.

These requirements should be translated into observable behaviour. A reviewer should be able to provide an input, follow the user interface, and confirm the expected output without needing to infer what the system intended to do. Requirements that cannot be tested should be clarified or rewritten. This also helps the team decide which features are essential for the current submission and which belong in future work.

The project should be judged against its stated scope rather than against features found in commercial research platforms. A controlled academic prototype can demonstrate sound algorithm selection and modular design even when it does not provide live indexing or a very large corpus. Clear boundaries make the report more credible and help future development focus on improvements that solve a defined problem.

The system should also be assessed from the perspective of the person using it. A technically correct output is not enough if the user cannot understand why it appeared or what action to take next. Labels, examples, sensible defaults, and clear empty states reduce confusion. Where the system makes an approximate decision, such as suggesting a spelling correction or a related paper, the interface should communicate that uncertainty and let the user inspect the underlying record before relying on it.

Finally, the implementation should remain aligned with the evidence available in the project. Features that are implemented should be supported by a demonstration or test; features that are planned should be described as future work. This distinction is especially important in an academic report, where clear reasoning and honest limitations are stronger than unsupported claims. Keeping the documentation, source code, and demonstration consistent will make the project easier to evaluate and extend.

A final consideration is maintainability over time. As new records and search options are added, the team should keep a short change log, update the test cases, and review whether the current data structures still suit the workload. Small, well-documented changes are easier to verify than large rewrites. The project can therefore grow in stages: establish correct results first, improve the user experience next, and only then optimise measured bottlenecks or introduce advanced discovery techniques.

2.4 Software Requirements

The project materials identify Java, Spring Boot, Maven, React.js, Vite, JavaScript, HTML, CSS, Node.js, and npm as the main software technologies. Java hosts the backend algorithms and service logic. Spring Boot provides the application framework, dependency injection, controller routing, and REST response handling. Maven manages Java dependencies and build tasks. React supports reusable interface components, while Vite provides a development server and frontend build tooling.

Node.js supplies the runtime used by the JavaScript toolchain, and npm installs frontend packages and runs scripts. HTML describes the interface structure and CSS controls presentation. A browser is used to interact with the application. A code editor or IDE assists with implementation and debugging. Exact versions should be checked in the project configuration so that team members use compatible environments.

Installation should be performed in a repeatable order: verify Java and Maven or the Maven wrapper, install Node.js and npm, install frontend dependencies, configure the backend port and allowed frontend origin, and start the services. The backend and frontend may use different local ports, so the API base URL and CORS configuration must agree. Environment-specific settings should be documented rather than hidden in a developer's machine.

These requirements should be translated into observable behaviour. A reviewer should be able to provide an input, follow the user interface, and confirm the expected output without needing to infer what the system intended to do. Requirements that cannot be tested should be clarified or rewritten. This also helps the team decide which features are essential for the current submission and which belong in future work.

The project should be judged against its stated scope rather than against features found in commercial research platforms. A controlled academic prototype can demonstrate sound algorithm selection and modular design even when it does not provide live indexing or a very large corpus. Clear boundaries make the report more credible and help future development focus on improvements that solve a defined problem.

The system should also be assessed from the perspective of the person using it. A technically correct output is not enough if the user cannot understand why it appeared or what action to take next. Labels, examples, sensible defaults, and clear empty states reduce confusion. Where the system makes an approximate decision, such as suggesting a spelling correction or a related paper, the interface should communicate that uncertainty and let the user inspect the underlying record before relying on it.

Finally, the implementation should remain aligned with the evidence available in the project. Features that are implemented should be supported by a demonstration or test; features that are planned should be described as future work. This distinction is especially important in an academic report, where clear reasoning and honest limitations are stronger than unsupported claims. Keeping the documentation, source code, and demonstration consistent will make the project easier to evaluate and extend.

A final consideration is maintainability over time. As new records and search options are added, the team should keep a short change log, update the test cases, and review whether the current data structures still suit the workload. Small, well-documented changes are easier to verify than large rewrites. The project can therefore grow in stages: establish correct results first, improve the user experience next, and only then optimise measured bottlenecks or introduce advanced discovery techniques.

CHAPTER 3 — SYSTEM DESIGN

3.1 High-Level Architecture

Research Connect follows a layered full-stack architecture. The presentation layer is a React.js application built with Vite. It collects user input, manages interface state, displays search results, and offers navigation to details, similarity results, statistics, and citation information. The frontend communicates with the backend through HTTP requests to REST endpoints and receives structured responses, typically represented as JSON.

The application layer is implemented using Java Spring Boot. Controllers receive requests and validate parameters. Service classes coordinate operations and call the appropriate algorithm or repository method. The algorithm layer contains focused implementations such as KMP, Aho–Corasick, edit distance, suffix-array/LCP analysis, and graph traversal. The data layer provides the paper corpus and relationships. Hash maps and inverted indexes can be built over this corpus to support efficient retrieval.

The layers are separated to keep responsibilities clear. The frontend should not reimplement backend matching logic, and the algorithm classes should not depend on visual components. This separation improves testability and permits a future database migration or interface redesign with fewer changes. A well-defined architecture also makes debugging easier because a problem can be traced through request handling, service logic, algorithm output, and response rendering.

The practical value of this section is that it connects the project design to decisions that can be verified during implementation. Each field, algorithm, or workflow should have a defined purpose, and its output should be understandable to the next module that uses it. Explicit assumptions reduce ambiguity for both developers and reviewers. When an assumption changes, related tests and documentation should be updated together.

A good next step is to validate this part of the system with representative examples, including missing data and unexpected input. The team should note what works, what fails, and what remains unimplemented. This evidence-based approach keeps the final report aligned with the source code and gives a practical starting point for future improvements.

The system should also be assessed from the perspective of the person using it. A technically correct output is not enough if the user cannot understand why it appeared or what action to take next. Labels, examples, sensible defaults, and clear empty states reduce confusion. Where the system makes an approximate decision, such as suggesting a spelling correction or a related paper, the interface should communicate that uncertainty and let the user inspect the underlying record before relying on it.

Finally, the implementation should remain aligned with the evidence available in the project. Features that are implemented should be supported by a demonstration or test; features that are planned should be described as future work. This distinction is especially important in an academic report, where clear reasoning and honest limitations are stronger than unsupported claims. Keeping the documentation, source code, and demonstration consistent will make the project easier to evaluate and extend.

A final consideration is maintainability over time. As new records and search options are added, the team should keep a short change log, update the test cases, and review whether the current data structures still suit the workload. Small, well-documented changes are easier to verify than large rewrites. The project can therefore grow in stages: establish correct results first, improve the user experience next, and only then optimise measured bottlenecks or introduce advanced discovery techniques.

3.2 Presentation Layer

The presentation layer is responsible for the user's experience. A search component captures the query and any selected search mode. Filter controls allow selection of publication year or category. Result cards show the most useful metadata at a glance, while a details view presents additional fields for a selected paper. Separate components can be used for recommendations, statistics, and citation exploration so that each feature remains manageable.

React state should represent the current query, active filters, loading status, results, error messages, and selected paper. When a user changes a filter or submits a new query, the relevant state is updated and the application sends a request to the backend. Loading indicators prevent the page from appearing unresponsive. Empty states should explain that no records matched the current query and suggest removing filters or trying different terms.

Good interface design is not only decorative. It helps users interpret the difference between exact matches and approximate suggestions. Labels should identify why a paper appears, for example "keyword match," "similar text," or "connected by citation." The interface should also avoid presenting synthetic sample records as authentic publications. Consistent spacing, readable typography, keyboard-accessible controls, and meaningful error messages improve usability.

A clear separation of responsibilities also helps the team work in parallel. One member can improve the interface while another tests backend services, provided both agree on request fields and response structures. The contract should state required fields, optional fields, error cases, and the meaning of returned values. This avoids fragile assumptions in which one component expects a field that another component does not provide.

For maintenance, keep configuration outside business logic and avoid duplicating endpoint addresses or validation rules across multiple components. Use consistent naming, small methods, and logs that identify the failed operation without exposing unnecessary internal details. A setup guide should explain how to install dependencies, configure local ports, start each service, and diagnose common connection problems. These practices make the project easier to reproduce during assessment.

The system should also be assessed from the perspective of the person using it. A technically correct output is not enough if the user cannot understand why it appeared or what action to take next. Labels, examples, sensible defaults, and clear empty states reduce confusion. Where the system makes an approximate decision, such as suggesting a spelling correction or a related paper, the interface should communicate that uncertainty and let the user inspect the underlying record before relying on it.

Finally, the implementation should remain aligned with the evidence available in the project. Features that are implemented should be supported by a demonstration or test; features that are planned should be described as future work. This distinction is especially important in an academic report, where clear reasoning and honest limitations are stronger than unsupported claims. Keeping the documentation, source code, and demonstration consistent will make the project easier to evaluate and extend.

A final consideration is maintainability over time. As new records and search options are added, the team should keep a short change log, update the test cases, and review whether the current data structures still suit the workload. Small, well-documented changes are easier to verify than large rewrites. The project can therefore grow in stages: establish correct results first, improve the user experience next, and only then optimise measured bottlenecks or introduce advanced discovery techniques.

3.3 Application and Service Layer

The Spring Boot application layer receives requests from the frontend and coordinates the work needed to answer them. Controllers should remain relatively thin: parse the request, validate required fields, call a service, and return the response. Services hold the application workflow, such as retrieving candidate papers, applying filters, running a matching algorithm, and preparing result objects. This separation makes it possible to test business logic without launching a browser.

An algorithm service should define the expected input and output of each operation. For example, a KMP method can accept text and a pattern and return match positions. A multi-pattern service can accept a list of terms and return occurrences grouped by term. A similarity service can accept a paper identifier and a limit, then return candidate records with a transparent similarity explanation. The exact response structure should be documented and kept stable for the frontend.

Validation belongs at the boundary. Empty queries, extremely long inputs, unsupported filter values, and unknown paper identifiers should be handled deliberately. Exceptions should be translated into appropriate HTTP status codes and readable messages. Logging can record technical failures, but should avoid unnecessary personal or sensitive information. These practices improve reliability and make the application easier to extend.

A clear separation of responsibilities also helps the team work in parallel. One member can improve the interface while another tests backend services, provided both agree on request fields and response structures. The contract should state required fields, optional fields, error cases, and the meaning of returned values. This avoids fragile assumptions in which one component expects a field that another component does not provide.

For maintenance, keep configuration outside business logic and avoid duplicating endpoint addresses or validation rules across multiple components. Use consistent naming, small methods, and logs that identify the failed operation without exposing unnecessary internal details. A setup guide should explain how to install dependencies, configure local ports, start each service, and diagnose common connection problems. These practices make the project easier to reproduce during assessment.

The system should also be assessed from the perspective of the person using it. A technically correct output is not enough if the user cannot understand why it appeared or what action to take next. Labels, examples, sensible defaults, and clear empty states reduce confusion. Where the system makes an approximate decision, such as suggesting a spelling correction or a related paper, the interface should communicate that uncertainty and let the user inspect the underlying record before relying on it.

Finally, the implementation should remain aligned with the evidence available in the project. Features that are implemented should be supported by a demonstration or test; features that are planned should be described as future work. This distinction is especially important in an academic report, where clear reasoning and honest limitations are stronger than unsupported claims. Keeping the documentation, source code, and demonstration consistent will make the project easier to evaluate and extend.

A final consideration is maintainability over time. As new records and search options are added, the team should keep a short change log, update the test cases, and review whether the current data structures still suit the workload. Small, well-documented changes are easier to verify than large rewrites. The project can therefore grow in stages: establish correct results first, improve the user experience next, and only then optimise measured bottlenecks or introduce advanced discovery techniques.

3.4 Algorithm Layer

The algorithm layer provides the specialised computations used by Research Connect. KMP handles exact pattern matching within a string. Aho–Corasick handles several patterns in one scan by combining a trie with failure links. Levenshtein edit distance measures the minimum number of single-character insertions, deletions, and substitutions required to transform one string into another. These techniques address different query needs and should not be treated as interchangeable.

Suffix arrays store suffix starting positions in lexicographic order, and LCP values record the common-prefix length between adjacent suffixes. They can reveal repeated textual patterns, but the interpretation of those patterns needs care. Citation graphs model relationships between paper identifiers. BFS explores by increasing distance from a starting node, while DFS follows a path deeply before backtracking. Hash maps, sets, queues, arrays, tries, and inverted indexes support these algorithms.

Each algorithm should have unit tests for ordinary cases and edge cases. Empty text, empty patterns, repeated characters, overlapping matches, Unicode text, duplicated terms, cycles in graphs, and missing paper IDs can expose subtle bugs. Complexity analysis should state which quantities matter, such as text length, number of patterns, number of graph vertices and edges, or total corpus size.

The practical value of this section is that it connects the project design to decisions that can be verified during implementation. Each field, algorithm, or workflow should have a defined purpose, and its output should be understandable to the next module that uses it. Explicit assumptions reduce ambiguity for both developers and reviewers. When an assumption changes, related tests and documentation should be updated together.

A good next step is to validate this part of the system with representative examples, including missing data and unexpected input. The team should note what works, what fails, and what remains unimplemented. This evidence-based approach keeps the final report aligned with the source code and gives a practical starting point for future improvements.

The system should also be assessed from the perspective of the person using it. A technically correct output is not enough if the user cannot understand why it appeared or what action to take next. Labels, examples, sensible defaults, and clear empty states reduce confusion. Where the system makes an approximate decision, such as suggesting a spelling correction or a related paper, the interface should communicate that uncertainty and let the user inspect the underlying record before relying on it.

Finally, the implementation should remain aligned with the evidence available in the project. Features that are implemented should be supported by a demonstration or test; features that are planned should be described as future work. This distinction is especially important in an academic report, where clear reasoning and honest limitations are stronger than unsupported claims. Keeping the documentation, source code, and demonstration consistent will make the project easier to evaluate and extend.

A final consideration is maintainability over time. As new records and search options are added, the team should keep a short change log, update the test cases, and review whether the current data structures still suit the workload. Small, well-documented changes are easier to verify than large rewrites. The project can therefore grow in stages: establish correct results first, improve the user experience next, and only then optimise measured bottlenecks or introduce advanced discovery techniques.

3.5 Data Layer and Paper Model

A paper record needs a stable identifier and fields that support search and display. Based on the project description, relevant fields include title, author, publication year, category, content, and citation relationships. Additional optional fields may include abstract, source URL, venue, DOI, and keywords if they are available in the actual corpus. A missing field should be represented consistently rather than filled with invented information.

The data model should separate descriptive metadata from large text content when the corpus grows. A paper identifier can map to a Paper object through a HashMap, providing convenient average constant-time lookup under normal hashing assumptions. A list or repository abstraction can support iteration and filtering. Citation edges should refer to stable identifiers so that the graph remains valid if display titles change.

Data loading is a critical step because all downstream algorithms rely on it. The loader should check required fields, normalise whitespace, handle encoding, reject or report duplicate identifiers, and record malformed entries. If records are synthetic, they should carry a marker or be described clearly in the interface. Good data quality improves search accuracy more than algorithm choice alone can compensate for poor metadata.

A clear separation of responsibilities also helps the team work in parallel. One member can improve the interface while another tests backend services, provided both agree on request fields and response structures. The contract should state required fields, optional fields, error cases, and the meaning of returned values. This avoids fragile assumptions in which one component expects a field that another component does not provide.

For maintenance, keep configuration outside business logic and avoid duplicating endpoint addresses or validation rules across multiple components. Use consistent naming, small methods, and logs that identify the failed operation without exposing unnecessary internal details. A setup guide should explain how to install dependencies, configure local ports, start each service, and diagnose common connection problems. These practices make the project easier to reproduce during assessment.

The system should also be assessed from the perspective of the person using it. A technically correct output is not enough if the user cannot understand why it appeared or what action to take next. Labels, examples, sensible defaults, and clear empty states reduce confusion. Where the system makes an approximate decision, such as suggesting a spelling correction or a related paper, the interface should communicate that uncertainty and let the user inspect the underlying record before relying on it.

Finally, the implementation should remain aligned with the evidence available in the project. Features that are implemented should be supported by a demonstration or test; features that are planned should be described as future work. This distinction is especially important in an academic report, where clear reasoning and honest limitations are stronger than unsupported claims. Keeping the documentation, source code, and demonstration consistent will make the project easier to evaluate and extend.

A final consideration is maintainability over time. As new records and search options are added, the team should keep a short change log, update the test cases, and review whether the current data structures still suit the workload. Small, well-documented changes are easier to verify than large rewrites. The project can therefore grow in stages: establish correct results first, improve the user experience next, and only then optimise measured bottlenecks or introduce advanced discovery techniques.

3.6 Data Flow and Request Lifecycle

A typical search begins in the browser when a user enters a term and presses Search. The frontend validates basic input, sets a loading state, and sends a request to a search endpoint. The controller reads the parameters and calls the relevant service. The service uses an index to identify candidate records, applies exact or approximate matching as appropriate, and applies selected filters. The resulting records are converted to a response format and returned to the browser.

The frontend checks whether the request succeeded. If it did, it updates the result list and clears the loading state. If it failed, the interface displays a helpful error message and preserves enough context for the user to retry. For details, similarity, and citation requests, the same general pattern applies but the endpoint and service differ. Keeping a consistent request lifecycle reduces duplicated code and makes behaviour easier to test.

For troubleshooting, it is useful to identify the stage where a failure occurs. A browser network panel can show the URL, status code, and response body. Backend logs can show whether a controller was reached and whether an algorithm raised an exception. Unit tests can isolate the algorithm from the HTTP layer. This layered diagnosis avoids making random changes to the interface when the underlying problem is in data loading or backend configuration.

A clear separation of responsibilities also helps the team work in parallel. One member can improve the interface while another tests backend services, provided both agree on request fields and response structures. The contract should state required fields, optional fields, error cases, and the meaning of returned values. This avoids fragile assumptions in which one component expects a field that another component does not provide.

For maintenance, keep configuration outside business logic and avoid duplicating endpoint addresses or validation rules across multiple components. Use consistent naming, small methods, and logs that identify the failed operation without exposing unnecessary internal details. A setup guide should explain how to install dependencies, configure local ports, start each service, and diagnose common connection problems. These practices make the project easier to reproduce during assessment.

The system should also be assessed from the perspective of the person using it. A technically correct output is not enough if the user cannot understand why it appeared or what action to take next. Labels, examples, sensible defaults, and clear empty states reduce confusion. Where the system makes an approximate decision, such as suggesting a spelling correction or a related paper, the interface should communicate that uncertainty and let the user inspect the underlying record before relying on it.

Finally, the implementation should remain aligned with the evidence available in the project. Features that are implemented should be supported by a demonstration or test; features that are planned should be described as future work. This distinction is especially important in an academic report, where clear reasoning and honest limitations are stronger than unsupported claims. Keeping the documentation, source code, and demonstration consistent will make the project easier to evaluate and extend.

A final consideration is maintainability over time. As new records and search options are added, the team should keep a short change log, update the test cases, and review whether the current data structures still suit the workload. Small, well-documented changes are easier to verify than large rewrites. The project can therefore grow in stages: establish correct results first, improve the user experience next, and only then optimise measured bottlenecks or introduce advanced discovery techniques.

3.8 Data Structures Used in the Project

A HashMap associates a key, such as a paper ID or token, with a value. It is useful for retrieving a paper object without scanning a list every time. A HashSet stores unique values and is useful for preventing duplicate candidate IDs when a word appears more than once in an index. Both structures are commonly used in retrieval systems because their average lookup and insertion costs are efficient under normal hash distribution.

A trie stores a vocabulary as a tree of character transitions. Words sharing a prefix also share the path for that prefix, making tries suitable for prefix search and autocomplete. A queue supports level-order processing in BFS and can be used while building Aho–Corasick failure links. Arrays store indexed values such as KMP prefix lengths, dynamic-programming rows, suffix positions, and LCP values.

A directed graph represents citations. Each vertex is a paper and each directed edge indicates a citation relationship according to the chosen convention. An inverted index maps a term to the identifiers of papers that contain it. These structures are complementary: the paper map supports record lookup, the inverted index supports candidate generation, and the citation graph supports relationship exploration. Choosing a structure depends on the operation the application must perform.

The practical value of this section is that it connects the project design to decisions that can be verified during implementation. Each field, algorithm, or workflow should have a defined purpose, and its output should be understandable to the next module that uses it. Explicit assumptions reduce ambiguity for both developers and reviewers. When an assumption changes, related tests and documentation should be updated together.

A good next step is to validate this part of the system with representative examples, including missing data and unexpected input. The team should note what works, what fails, and what remains unimplemented. This evidence-based approach keeps the final report aligned with the source code and gives a practical starting point for future improvements.

The system should also be assessed from the perspective of the person using it. A technically correct output is not enough if the user cannot understand why it appeared or what action to take next. Labels, examples, sensible defaults, and clear empty states reduce confusion. Where the system makes an approximate decision, such as suggesting a spelling correction or a related paper, the interface should communicate that uncertainty and let the user inspect the underlying record before relying on it.

Finally, the implementation should remain aligned with the evidence available in the project. Features that are implemented should be supported by a demonstration or test; features that are planned should be described as future work. This distinction is especially important in an academic report, where clear reasoning and honest limitations are stronger than unsupported claims. Keeping the documentation, source code, and demonstration consistent will make the project easier to evaluate and extend.

A final consideration is maintainability over time. As new records and search options are added, the team should keep a short change log, update the test cases, and review whether the current data structures still suit the workload. Small, well-documented changes are easier to verify than large rewrites. The project can therefore grow in stages: establish correct results first, improve the user experience next, and only then optimise measured bottlenecks or introduce advanced discovery techniques.

CHAPTER 4 — ALGORITHMS AND IMPLEMENTATION

4.1 Corpus Loading and Normalisation

The loading module reads the controlled corpus and converts each record into an internal paper representation. The loader should ensure that the record has an identifier and the metadata fields expected by the application. It should preserve the original title and author spelling for display while preparing a normalised representation for matching. Normalisation can include lowercasing for case-insensitive search, standardising whitespace, and treating punctuation consistently where appropriate.

Care is needed not to destroy information. Removing every punctuation mark may change technical identifiers, programming language names, or hyphenated terms. Unicode characters can have multiple representations, so a robust system should define how it normalises them. The display version and search version may be stored separately. The loader should also record which documents lack full text, since metadata-only records cannot participate in content-level algorithms in the same way.

After loading, the system can build its supporting indexes. A paper map associates IDs with paper objects. The inverted index associates tokens with sets of IDs. A trie may be built from vocabulary terms. Citation edges are collected into graph adjacency lists. Building these structures once and updating them when the corpus changes is usually more efficient than rebuilding them for every user query.

The practical value of this section is that it connects the project design to decisions that can be verified during implementation. Each field, algorithm, or workflow should have a defined purpose, and its output should be understandable to the next module that uses it. Explicit assumptions reduce ambiguity for both developers and reviewers. When an assumption changes, related tests and documentation should be updated together.

A good next step is to validate this part of the system with representative examples, including missing data and unexpected input. The team should note what works, what fails, and what remains unimplemented. This evidence-based approach keeps the final report aligned with the source code and gives a practical starting point for future improvements.

The system should also be assessed from the perspective of the person using it. A technically correct output is not enough if the user cannot understand why it appeared or what action to take next. Labels, examples, sensible defaults, and clear empty states reduce confusion. Where the system makes an approximate decision, such as suggesting a spelling correction or a related paper, the interface should communicate that uncertainty and let the user inspect the underlying record before relying on it.

Finally, the implementation should remain aligned with the evidence available in the project. Features that are implemented should be supported by a demonstration or test; features that are planned should be described as future work. This distinction is especially important in an academic report, where clear reasoning and honest limitations are stronger than unsupported claims. Keeping the documentation, source code, and demonstration consistent will make the project easier to evaluate and extend.

A final consideration is maintainability over time. As new records and search options are added, the team should keep a short change log, update the test cases, and review whether the current data structures still suit the workload. Small, well-documented changes are easier to verify than large rewrites. The project can therefore grow in stages: establish correct results first, improve the user experience next, and only then optimise measured bottlenecks or introduce advanced discovery techniques.

4.2 KMP Algorithm: Purpose and Principle

The Knuth–Morris–Pratt algorithm searches for an exact pattern inside a text. A naive method may restart the pattern comparison from its beginning after every mismatch, repeating work when the text contains recurring character sequences. KMP avoids much of this repetition by preprocessing the pattern into an LPS array. LPS means “longest proper prefix which is also a suffix” for each prefix of the pattern.

During the search, two positions are tracked: one in the text and one in the pattern. When characters match, both positions advance. When a mismatch occurs after some pattern characters have matched, KMP uses the LPS value to move the pattern position to the longest prefix that could still match the text already examined. The text position does not need to move backwards. If the pattern position is zero, the algorithm advances the text position.

The algorithm takes $O(N + M)$ time for text length N and pattern length M , including pattern preprocessing, and uses $O(M)$ additional space for the LPS array. This makes it useful for exact keyword matching in paper text. The application should define how empty patterns are handled and whether matching is case-sensitive. For user-friendly search, normalisation is often applied consistently to both the query and indexed text.

In practical use, the algorithm must be treated as one stage in a complete retrieval pipeline. The input should be normalised consistently, the output should be mapped back to paper identifiers, and duplicate candidates should be removed where the user expects one result per paper. Developers should document whether matching is case-sensitive, how positions are counted, and how empty input is handled. These small decisions affect both correctness and the way results are explained in the interface.

A useful verification method is to compare the implementation with a simple reference method on a small dataset whose answer can be checked manually. Include ordinary examples as well as boundary cases, then repeat the tests after any refactoring. Complexity analysis predicts how work grows, but measured performance also depends on data layout, allocation, text length, and the number of matches returned. This combination of correctness tests and measurement makes the feature easier to trust and maintain.

The system should also be assessed from the perspective of the person using it. A technically correct output is not enough if the user cannot understand why it appeared or what action to take next. Labels, examples, sensible defaults, and clear empty states reduce confusion. Where the system makes an approximate decision, such as suggesting a spelling correction or a related paper, the interface should communicate that uncertainty and let the user inspect the underlying record before relying on it.

Finally, the implementation should remain aligned with the evidence available in the project. Features that are implemented should be supported by a demonstration or test; features that are planned should be described as future work. This distinction is especially important in an academic report, where clear reasoning and honest limitations are stronger than unsupported claims. Keeping the documentation, source code, and demonstration consistent will make the project easier to evaluate and extend.

A final consideration is maintainability over time. As new records and search options are added, the team should keep a short change log, update the test cases, and review whether the current data structures still suit the workload. Small, well-documented changes are easier to verify than large rewrites. The project can therefore grow in stages: establish correct results first, improve the user experience next, and only then optimise measured bottlenecks or introduce advanced discovery techniques.

4.3 KMP Example and Edge Cases

Consider the text “research papers support research” and the pattern “research.” KMP preprocesses the pattern, then compares it from left to right. If a mismatch happens after part of the word has matched, the LPS array indicates whether a shorter prefix of “research” can be retained as a possible match. This matters for patterns with repeated structure, because the algorithm can reuse information rather than compare the same text characters again.

Useful test cases include a pattern that appears at the beginning, a pattern that appears at the end, a pattern that appears multiple times, a pattern that does not occur, a pattern longer than the text, and a pattern containing repeated characters such as “ababaca.” Tests should also cover case differences and punctuation if the application supports normalised search. Expected positions should be defined using one consistent indexing convention, usually zero-based internally.

KMP finds exact character sequences; it does not understand synonyms, related concepts, or the meaning of a paragraph. If the user searches “AI” and a paper uses “artificial intelligence,” KMP alone will not infer equivalence. That limitation is why the project combines several techniques. Search results should make the matching method clear so that users do not confuse exact matching with semantic discovery.

In practical use, the algorithm must be treated as one stage in a complete retrieval pipeline. The input should be normalised consistently, the output should be mapped back to paper identifiers, and duplicate candidates should be removed where the user expects one result per paper. Developers should document whether matching is case-sensitive, how positions are counted, and how empty input is handled. These small decisions affect both correctness and the way results are explained in the interface.

A useful verification method is to compare the implementation with a simple reference method on a small dataset whose answer can be checked manually. Include ordinary examples as well as boundary cases, then repeat the tests after any refactoring. Complexity analysis predicts how work grows, but measured performance also depends on data layout, allocation, text length, and the number of matches returned. This combination of correctness tests and measurement makes the feature easier to trust and maintain.

The system should also be assessed from the perspective of the person using it. A technically correct output is not enough if the user cannot understand why it appeared or what action to take next. Labels, examples, sensible defaults, and clear empty states reduce confusion. Where the system makes an approximate decision, such as suggesting a spelling correction or a related paper, the interface should communicate that uncertainty and let the user inspect the underlying record before relying on it.

Finally, the implementation should remain aligned with the evidence available in the project. Features that are implemented should be supported by a demonstration or test; features that are planned should be described as future work. This distinction is especially important in an academic report, where clear reasoning and honest limitations are stronger than unsupported claims. Keeping the documentation, source code, and demonstration consistent will make the project easier to evaluate and extend.

A final consideration is maintainability over time. As new records and search options are added, the team should keep a short change log, update the test cases, and review whether the current data structures still suit the workload. Small, well-documented changes are easier to verify than large rewrites. The project can therefore grow in stages: establish correct results first, improve the user experience next, and only then optimise measured bottlenecks or introduce advanced discovery techniques.

4.4 Aho–Corasick: Multiple-Pattern Search

Aho–Corasick is designed to search for many patterns in a text simultaneously. It first inserts the patterns into a trie. Each trie edge represents a character transition, and terminal nodes indicate that a complete search term has been found. The algorithm then computes failure links, which indicate where to continue when the current transition cannot be followed. These links allow the search to reuse suffix information across patterns.

Failure links are commonly built with a queue-based breadth-first process. Nodes close to the root are processed first, allowing a child’s failure target to be determined from its parent’s failure target. During text scanning, the algorithm follows trie transitions and failure links until it can consume the next character. When a terminal state is reached, the associated keyword or keywords are reported. Several terms can therefore be checked during one pass over the text.

If the total length of all patterns is P , the text length is N , and Z matches are reported, the standard complexity is commonly expressed as $O(P + N + Z)$, subject to implementation details. The approach is useful when a user supplies several research concepts or when the application checks a document against a fixed vocabulary. Duplicate patterns, overlapping matches, case normalisation, and output ordering should be handled consistently.

In practical use, the algorithm must be treated as one stage in a complete retrieval pipeline. The input should be normalised consistently, the output should be mapped back to paper identifiers, and duplicate candidates should be removed where the user expects one result per paper. Developers should document whether matching is case-sensitive, how positions are counted, and how empty input is handled. These small decisions affect both correctness and the way results are explained in the interface.

A useful verification method is to compare the implementation with a simple reference method on a small dataset whose answer can be checked manually. Include ordinary examples as well as boundary cases, then repeat the tests after any refactoring. Complexity analysis predicts how work grows, but measured performance also depends on data layout, allocation, text length, and the number of matches returned. This combination of correctness tests and measurement makes the feature easier to trust and maintain.

The system should also be assessed from the perspective of the person using it. A technically correct output is not enough if the user cannot understand why it appeared or what action to take next. Labels, examples, sensible defaults, and clear empty states reduce confusion. Where the system makes an approximate decision, such as suggesting a spelling correction or a related paper, the interface should communicate that uncertainty and let the user inspect the underlying record before relying on it.

Finally, the implementation should remain aligned with the evidence available in the project. Features that are implemented should be supported by a demonstration or test; features that are planned should be described as future work. This distinction is especially important in an academic report, where clear reasoning and honest limitations are stronger than unsupported claims. Keeping the documentation, source code, and demonstration consistent will make the project easier to evaluate and extend.

A final consideration is maintainability over time. As new records and search options are added, the team should keep a short change log, update the test cases, and review whether the current data structures still suit the workload. Small, well-documented changes are easier to verify than large rewrites. The project can therefore grow in stages: establish correct results first, improve the user experience next, and only then optimise measured bottlenecks or introduce advanced discovery techniques.

4.6 Levenshtein Edit Distance

Levenshtein edit distance measures the minimum number of single-character insertions, deletions, and substitutions needed to transform one string into another. It is useful when a user enters a misspelled technical term. For example, “algorithm” is close to “algoritm” because one insertion can make the strings equal. Edit distance provides a numeric measure of spelling difference rather than a binary exact-match result.

The classic dynamic-programming method constructs a table D where $D[i][j]$ is the minimum edit distance between the first i characters of one string and the first j characters of another. The first row and first column represent converting to or from an empty string. Each remaining cell takes the minimum of deletion, insertion, and substitution costs. The final cell gives the distance between the complete strings.

For strings of lengths M and N , the full table requires $O(MN)$ time and space. If only the distance is needed, memory can be reduced to two rows. For search suggestions, the application compares the query with vocabulary terms and ranks candidates by distance, optionally applying a threshold based on query length. A small distance is evidence of spelling similarity, not proof that the candidate is the intended concept.

In practical use, the algorithm must be treated as one stage in a complete retrieval pipeline. The input should be normalised consistently, the output should be mapped back to paper identifiers, and duplicate candidates should be removed where the user expects one result per paper. Developers should document whether matching is case-sensitive, how positions are counted, and how empty input is handled. These small decisions affect both correctness and the way results are explained in the interface.

A useful verification method is to compare the implementation with a simple reference method on a small dataset whose answer can be checked manually. Include ordinary examples as well as boundary cases, then repeat the tests after any refactoring. Complexity analysis predicts how work grows, but measured performance also depends on data layout, allocation, text length, and the number of matches returned. This combination of correctness tests and measurement makes the feature easier to trust and maintain.

The system should also be assessed from the perspective of the person using it. A technically correct output is not enough if the user cannot understand why it appeared or what action to take next. Labels, examples, sensible defaults, and clear empty states reduce confusion. Where the system makes an approximate decision, such as suggesting a spelling correction or a related paper, the interface should communicate that uncertainty and let the user inspect the underlying record before relying on it.

Finally, the implementation should remain aligned with the evidence available in the project. Features that are implemented should be supported by a demonstration or test; features that are planned should be described as future work. This distinction is especially important in an academic report, where clear reasoning and honest limitations are stronger than unsupported claims. Keeping the documentation, source code, and demonstration consistent will make the project easier to evaluate and extend.

A final consideration is maintainability over time. As new records and search options are added, the team should keep a short change log, update the test cases, and review whether the current data structures still suit the workload. Small, well-documented changes are easier to verify than large rewrites. The project can therefore grow in stages: establish correct results first, improve the user experience next, and only then optimise measured bottlenecks or introduce advanced discovery techniques.

4.8 Suffix Array

A suffix of a string is the substring that begins at a particular position and continues to the end. A suffix array stores the starting positions of all suffixes in sorted lexicographic order. For example, the suffixes of “banana” begin at positions 0 through 5; sorting those suffixes gives an ordering based on their character sequences. The suffix array makes it easier to reason about repeated substrings and to locate patterns using sorted suffixes.

Suffix arrays are often more memory-efficient than pointer-heavy suffix trees and are useful in text indexing. Construction algorithms vary in complexity. A simple sorting-based implementation may be adequate for small inputs but can become expensive for large texts; more advanced construction methods improve scalability. The implementation should document the chosen approach and the expected size of text it can handle.

In Research Connect, suffix-array analysis is described as part of the similarity feature. Text from papers may be combined with separators so that patterns do not accidentally cross document boundaries. The separator strategy must be chosen carefully, and suffix positions must be mapped back to their source paper and local offset. Without this mapping, a common substring can be detected but not reliably attributed to the documents that contain it.

In practical use, the algorithm must be treated as one stage in a complete retrieval pipeline. The input should be normalised consistently, the output should be mapped back to paper identifiers, and duplicate candidates should be removed where the user expects one result per paper. Developers should document whether matching is case-sensitive, how positions are counted, and how empty input is handled. These small decisions affect both correctness and the way results are explained in the interface.

A useful verification method is to compare the implementation with a simple reference method on a small dataset whose answer can be checked manually. Include ordinary examples as well as boundary cases, then repeat the tests after any refactoring. Complexity analysis predicts how work grows, but measured performance also depends on data layout, allocation, text length, and the number of matches returned. This combination of correctness tests and measurement makes the feature easier to trust and maintain.

The system should also be assessed from the perspective of the person using it. A technically correct output is not enough if the user cannot understand why it appeared or what action to take next. Labels, examples, sensible defaults, and clear empty states reduce confusion. Where the system makes an approximate decision, such as suggesting a spelling correction or a related paper, the interface should communicate that uncertainty and let the user inspect the underlying record before relying on it.

Finally, the implementation should remain aligned with the evidence available in the project. Features that are implemented should be supported by a demonstration or test; features that are planned should be described as future work. This distinction is especially important in an academic report, where clear reasoning and honest limitations are stronger than unsupported claims. Keeping the documentation, source code, and demonstration consistent will make the project easier to evaluate and extend.

A final consideration is maintainability over time. As new records and search options are added, the team should keep a short change log, update the test cases, and review whether the current data structures still suit the workload. Small, well-documented changes are easier to verify than large rewrites. The project can therefore grow in stages: establish correct results first, improve the user experience next, and only then optimise measured bottlenecks or introduce advanced discovery techniques.

4.9 Longest Common Prefix (LCP)

The LCP array stores the length of the longest common prefix between neighbouring suffixes in suffix-array order. When two adjacent suffixes share a long prefix, the corresponding LCP value indicates a repeated substring. Across a collection of documents, repeated fragments can be used as evidence that two records contain overlapping text. The exact interpretation depends on how the documents and separators were encoded.

Suffix arrays and LCP arrays are complementary. Sorting suffixes groups similar beginnings together, and the LCP array records how much they overlap. An implementation can inspect large LCP values and map those positions to paper identifiers. It should exclude separators and very common boilerplate where appropriate, since phrases such as “in this paper” may appear in many documents without indicating meaningful topical similarity.

Textual overlap is not the same as semantic similarity or recommendation quality. Two papers can discuss the same topic using different words, while unrelated papers may quote the same definition. A useful result should describe the basis of similarity and ideally combine text overlap with metadata, keywords, or citation relationships. The current project’s suffix-array/LCP approach is a demonstrative foundation that could later be complemented by more advanced text representations.

In practical use, the algorithm must be treated as one stage in a complete retrieval pipeline. The input should be normalised consistently, the output should be mapped back to paper identifiers, and duplicate candidates should be removed where the user expects one result per paper. Developers should document whether matching is case-sensitive, how positions are counted, and how empty input is handled. These small decisions affect both correctness and the way results are explained in the interface.

A useful verification method is to compare the implementation with a simple reference method on a small dataset whose answer can be checked manually. Include ordinary examples as well as boundary cases, then repeat the tests after any refactoring. Complexity analysis predicts how work grows, but measured performance also depends on data layout, allocation, text length, and the number of matches returned. This combination of correctness tests and measurement makes the feature easier to trust and maintain.

The system should also be assessed from the perspective of the person using it. A technically correct output is not enough if the user cannot understand why it appeared or what action to take next. Labels, examples, sensible defaults, and clear empty states reduce confusion. Where the system makes an approximate decision, such as suggesting a spelling correction or a related paper, the interface should communicate that uncertainty and let the user inspect the underlying record before relying on it.

Finally, the implementation should remain aligned with the evidence available in the project. Features that are implemented should be supported by a demonstration or test; features that are planned should be described as future work. This distinction is especially important in an academic report, where clear reasoning and honest limitations are stronger than unsupported claims. Keeping the documentation, source code, and demonstration consistent will make the project easier to evaluate and extend.

A final consideration is maintainability over time. As new records and search options are added, the team should keep a short change log, update the test cases, and review whether the current data structures still suit the workload. Small, well-documented changes are easier to verify than large rewrites. The project can therefore grow in stages: establish correct results first, improve the user experience next, and only then optimise measured bottlenecks or introduce advanced discovery techniques.

4.10 Similar-Paper Recommendation

The recommendation module starts from a selected paper and identifies other records that may be related. In the described project, common textual patterns from suffix-array and LCP analysis contribute to similarity information. A recommendation response should exclude the source paper itself, avoid duplicate records, and provide enough metadata for the user to decide whether to open a candidate.

A robust recommendation pipeline should distinguish candidate generation from ranking. Candidate generation finds papers with shared terms, overlapping patterns, or citation links. Ranking orders those candidates using a score. The score could combine several signals, but each signal needs a clear definition and appropriate normalisation. The report should not claim that a sophisticated weighted ranking exists unless the supplied code actually implements it.

Recommendations should be evaluated with sample queries and human review. Useful questions include whether the results are topically related, whether duplicates are suppressed, whether common boilerplate dominates the list, and whether the explanation helps users understand why a paper was suggested. Future work could include TF-IDF or BM25 lexical scoring, domain-specific synonyms, citation-based ranking, or embeddings. Those additions should be compared against a baseline rather than assumed to be better.

The practical value of this section is that it connects the project design to decisions that can be verified during implementation. Each field, algorithm, or workflow should have a defined purpose, and its output should be understandable to the next module that uses it. Explicit assumptions reduce ambiguity for both developers and reviewers. When an assumption changes, related tests and documentation should be updated together.

A good next step is to validate this part of the system with representative examples, including missing data and unexpected input. The team should note what works, what fails, and what remains unimplemented. This evidence-based approach keeps the final report aligned with the source code and gives a practical starting point for future improvements.

The system should also be assessed from the perspective of the person using it. A technically correct output is not enough if the user cannot understand why it appeared or what action to take next. Labels, examples, sensible defaults, and clear empty states reduce confusion. Where the system makes an approximate decision, such as suggesting a spelling correction or a related paper, the interface should communicate that uncertainty and let the user inspect the underlying record before relying on it.

Finally, the implementation should remain aligned with the evidence available in the project. Features that are implemented should be supported by a demonstration or test; features that are planned should be described as future work. This distinction is especially important in an academic report, where clear reasoning and honest limitations are stronger than unsupported claims. Keeping the documentation, source code, and demonstration consistent will make the project easier to evaluate and extend.

A final consideration is maintainability over time. As new records and search options are added, the team should keep a short change log, update the test cases, and review whether the current data structures still suit the workload. Small, well-documented changes are easier to verify than large rewrites. The project can therefore grow in stages: establish correct results first, improve the user experience next, and only then optimise measured bottlenecks or introduce advanced discovery techniques.

4.11 Citation Graph Representation

A citation network can be represented as a directed graph $G = (V, E)$. Each vertex in V represents a paper, and each directed edge in E represents a citation relation. Under the convention used in the project description, an edge $A \rightarrow B$ means that paper A cites paper B . Direction is important because “papers cited by A ” and “papers that cite A ” are different queries.

An adjacency list stores, for each paper, the identifiers of connected papers. It is often more memory-efficient than an adjacency matrix when the graph is sparse, as citation graphs commonly are. The graph should define how it handles references to records not present in the corpus. Such edges may be omitted, retained as external identifiers, or shown as unresolved references, but the choice must be consistent.

Graph construction depends on accurate citation metadata. A wrong identifier can create a false relationship, while missing references can make connected papers appear isolated. Duplicate edges should usually be removed. The system should also consider cycles, because citation relationships can form paths and cycles within a dataset. Graph traversal algorithms must track visited nodes so that the same paper is not processed indefinitely.

In practical use, the algorithm must be treated as one stage in a complete retrieval pipeline. The input should be normalised consistently, the output should be mapped back to paper identifiers, and duplicate candidates should be removed where the user expects one result per paper. Developers should document whether matching is case-sensitive, how positions are counted, and how empty input is handled. These small decisions affect both correctness and the way results are explained in the interface.

A useful verification method is to compare the implementation with a simple reference method on a small dataset whose answer can be checked manually. Include ordinary examples as well as boundary cases, then repeat the tests after any refactoring. Complexity analysis predicts how work grows, but measured performance also depends on data layout, allocation, text length, and the number of matches returned. This combination of correctness tests and measurement makes the feature easier to trust and maintain.

The system should also be assessed from the perspective of the person using it. A technically correct output is not enough if the user cannot understand why it appeared or what action to take next. Labels, examples, sensible defaults, and clear empty states reduce confusion. Where the system makes an approximate decision, such as suggesting a spelling correction or a related paper, the interface should communicate that uncertainty and let the user inspect the underlying record before relying on it.

Finally, the implementation should remain aligned with the evidence available in the project. Features that are implemented should be supported by a demonstration or test; features that are planned should be described as future work. This distinction is especially important in an academic report, where clear reasoning and honest limitations are stronger than unsupported claims. Keeping the documentation, source code, and demonstration consistent will make the project easier to evaluate and extend.

A final consideration is maintainability over time. As new records and search options are added, the team should keep a short change log, update the test cases, and review whether the current data structures still suit the workload. Small, well-documented changes are easier to verify than large rewrites. The project can therefore grow in stages: establish correct results first, improve the user experience next, and only then optimise measured bottlenecks or introduce advanced discovery techniques.

4.12 Breadth-First Search (BFS)

Breadth-First Search explores a graph in layers. It begins at a source vertex, marks it as visited, and places it in a queue. The algorithm repeatedly removes the next vertex from the queue and examines its neighbours. Each unvisited neighbour is marked and enqueued. Because the queue is first-in, first-out, vertices at a smaller number of edges from the source are processed before more distant vertices.

For a graph with V vertices and E edges represented by adjacency lists, BFS runs in $O(V + E)$ time. Its auxiliary memory includes the queue and visited information. In Research Connect, BFS can explore citation links level by level, helping a user inspect directly connected papers before more distant connections. If the graph is directed, traversal follows only edges in the selected direction unless the application explicitly builds a reverse graph.

Tests should include an isolated paper, a paper with several neighbours, a chain, a branching graph, a cycle, and disconnected components. The output order among neighbours can vary depending on adjacency-list order, so tests should either fix that order or verify the visited set rather than assume one arbitrary sequence. The interface should communicate what one traversal step means, such as one citation hop from the selected paper.

In practical use, the algorithm must be treated as one stage in a complete retrieval pipeline. The input should be normalised consistently, the output should be mapped back to paper identifiers, and duplicate candidates should be removed where the user expects one result per paper. Developers should document whether matching is case-sensitive, how positions are counted, and how empty input is handled. These small decisions affect both correctness and the way results are explained in the interface.

A useful verification method is to compare the implementation with a simple reference method on a small dataset whose answer can be checked manually. Include ordinary examples as well as boundary cases, then repeat the tests after any refactoring. Complexity analysis predicts how work grows, but measured performance also depends on data layout, allocation, text length, and the number of matches returned. This combination of correctness tests and measurement makes the feature easier to trust and maintain.

The system should also be assessed from the perspective of the person using it. A technically correct output is not enough if the user cannot understand why it appeared or what action to take next. Labels, examples, sensible defaults, and clear empty states reduce confusion. Where the system makes an approximate decision, such as suggesting a spelling correction or a related paper, the interface should communicate that uncertainty and let the user inspect the underlying record before relying on it.

Finally, the implementation should remain aligned with the evidence available in the project. Features that are implemented should be supported by a demonstration or test; features that are planned should be described as future work. This distinction is especially important in an academic report, where clear reasoning and honest limitations are stronger than unsupported claims. Keeping the documentation, source code, and demonstration consistent will make the project easier to evaluate and extend.

A final consideration is maintainability over time. As new records and search options are added, the team should keep a short change log, update the test cases, and review whether the current data structures still suit the workload. Small, well-documented changes are easier to verify than large rewrites. The project can therefore grow in stages: establish correct results first, improve the user experience next, and only then optimise measured bottlenecks or introduce advanced discovery techniques.

4.13 Depth-First Search (DFS)

Depth-First Search explores one path as far as possible before backtracking. It can be implemented recursively or with an explicit stack. Starting from a source node, DFS marks it as visited and then explores each unvisited neighbour. When no unvisited neighbour remains, it returns to the previous node and continues with another branch.

With adjacency lists, DFS also runs in $O(V + E)$ time. Recursive implementations are concise, but very deep graphs can risk stack overflow; an explicit stack can provide more control. DFS is useful for exploring connected citation paths, identifying reachable papers, and supporting future graph analyses. Like BFS, it requires visited-state tracking to handle cycles and avoid repeatedly exploring the same nodes.

BFS and DFS answer related but different exploration needs. BFS is useful when the number of citation steps from the source matters. DFS is useful when the goal is to follow paths deeply or perform structural traversals. Neither algorithm by itself determines whether a paper is academically important or relevant. They reveal connectivity; interpretation requires metadata and domain judgement.

In practical use, the algorithm must be treated as one stage in a complete retrieval pipeline. The input should be normalised consistently, the output should be mapped back to paper identifiers, and duplicate candidates should be removed where the user expects one result per paper. Developers should document whether matching is case-sensitive, how positions are counted, and how empty input is handled. These small decisions affect both correctness and the way results are explained in the interface.

A useful verification method is to compare the implementation with a simple reference method on a small dataset whose answer can be checked manually. Include ordinary examples as well as boundary cases, then repeat the tests after any refactoring. Complexity analysis predicts how work grows, but measured performance also depends on data layout, allocation, text length, and the number of matches returned. This combination of correctness tests and measurement makes the feature easier to trust and maintain.

The system should also be assessed from the perspective of the person using it. A technically correct output is not enough if the user cannot understand why it appeared or what action to take next. Labels, examples, sensible defaults, and clear empty states reduce confusion. Where the system makes an approximate decision, such as suggesting a spelling correction or a related paper, the interface should communicate that uncertainty and let the user inspect the underlying record before relying on it.

Finally, the implementation should remain aligned with the evidence available in the project. Features that are implemented should be supported by a demonstration or test; features that are planned should be described as future work. This distinction is especially important in an academic report, where clear reasoning and honest limitations are stronger than unsupported claims. Keeping the documentation, source code, and demonstration consistent will make the project easier to evaluate and extend.

A final consideration is maintainability over time. As new records and search options are added, the team should keep a short change log, update the test cases, and review whether the current data structures still suit the workload. Small, well-documented changes are easier to verify than large rewrites. The project can therefore grow in stages: establish correct results first, improve the user experience next, and only then optimise measured bottlenecks or introduce advanced discovery techniques.

4.14 Hashing and HashMap Lookup

Hashing maps a key to a location in a table using a hash function. In Java, HashMap provides key-value storage and is useful for mapping a paper identifier to a Paper object. Under normal conditions, lookup and insertion are expected to take constant average time, although performance depends on hashing behaviour, collisions, and implementation details. A HashSet uses hashing to store unique values and can help deduplicate paper identifiers.

Hash-based lookup complements an inverted index. The inverted index may return a set of paper IDs for a term; the HashMap then resolves each ID to the full record. This avoids scanning all paper objects merely to find the records associated with a term. Consistent identifiers are essential: if a citation edge or index entry uses a different spelling or format from the paper map, lookup may fail.

Hash structures do not automatically guarantee that the overall search is constant time. A query may return many candidates, and each candidate may require filtering, text matching, ranking, and response formatting. The total cost depends on all stages. The report should therefore describe HashMap as an efficient lookup component rather than claiming that every end-to-end search has $O(1)$ complexity.

In practical use, the algorithm must be treated as one stage in a complete retrieval pipeline. The input should be normalised consistently, the output should be mapped back to paper identifiers, and duplicate candidates should be removed where the user expects one result per paper. Developers should document whether matching is case-sensitive, how positions are counted, and how empty input is handled. These small decisions affect both correctness and the way results are explained in the interface.

A useful verification method is to compare the implementation with a simple reference method on a small dataset whose answer can be checked manually. Include ordinary examples as well as boundary cases, then repeat the tests after any refactoring. Complexity analysis predicts how work grows, but measured performance also depends on data layout, allocation, text length, and the number of matches returned. This combination of correctness tests and measurement makes the feature easier to trust and maintain.

The system should also be assessed from the perspective of the person using it. A technically correct output is not enough if the user cannot understand why it appeared or what action to take next. Labels, examples, sensible defaults, and clear empty states reduce confusion. Where the system makes an approximate decision, such as suggesting a spelling correction or a related paper, the interface should communicate that uncertainty and let the user inspect the underlying record before relying on it.

Finally, the implementation should remain aligned with the evidence available in the project. Features that are implemented should be supported by a demonstration or test; features that are planned should be described as future work. This distinction is especially important in an academic report, where clear reasoning and honest limitations are stronger than unsupported claims. Keeping the documentation, source code, and demonstration consistent will make the project easier to evaluate and extend.

A final consideration is maintainability over time. As new records and search options are added, the team should keep a short change log, update the test cases, and review whether the current data structures still suit the workload. Small, well-documented changes are easier to verify than large rewrites. The project can therefore grow in stages: establish correct results first, improve the user experience next, and only then optimise measured bottlenecks or introduce advanced discovery techniques.

4.15 Inverted Index

An inverted index maps each term to the documents in which that term occurs. For example, the entry “transformer” may point to Paper 1, Paper 5, and Paper 12. When the user searches for the term, the system can retrieve these candidate identifiers instead of examining every document in the corpus. This is a fundamental technique in information-retrieval systems because it shifts work from query time to index construction time.

Index construction requires tokenisation and normalisation. The implementation must decide how to handle punctuation, case, stop words, stemming, hyphenated terms, and duplicate occurrences. A posting list may store only document IDs or richer information such as term frequency and positions. The project description refers to an inverted index used for candidate retrieval; advanced scoring features should only be described as implemented if they exist in the code.

For a query containing multiple terms, the system can combine posting lists using set intersection, union, or a ranking strategy depending on whether all terms or any terms are required. Filters such as year and category can then be applied to the candidate set. Tests should compare indexed retrieval with a simple full scan on a small corpus to verify that no matching documents are accidentally omitted.

In practical use, the algorithm must be treated as one stage in a complete retrieval pipeline. The input should be normalised consistently, the output should be mapped back to paper identifiers, and duplicate candidates should be removed where the user expects one result per paper. Developers should document whether matching is case-sensitive, how positions are counted, and how empty input is handled. These small decisions affect both correctness and the way results are explained in the interface.

A useful verification method is to compare the implementation with a simple reference method on a small dataset whose answer can be checked manually. Include ordinary examples as well as boundary cases, then repeat the tests after any refactoring. Complexity analysis predicts how work grows, but measured performance also depends on data layout, allocation, text length, and the number of matches returned. This combination of correctness tests and measurement makes the feature easier to trust and maintain.

The system should also be assessed from the perspective of the person using it. A technically correct output is not enough if the user cannot understand why it appeared or what action to take next. Labels, examples, sensible defaults, and clear empty states reduce confusion. Where the system makes an approximate decision, such as suggesting a spelling correction or a related paper, the interface should communicate that uncertainty and let the user inspect the underlying record before relying on it.

Finally, the implementation should remain aligned with the evidence available in the project. Features that are implemented should be supported by a demonstration or test; features that are planned should be described as future work. This distinction is especially important in an academic report, where clear reasoning and honest limitations are stronger than unsupported claims. Keeping the documentation, source code, and demonstration consistent will make the project easier to evaluate and extend.

A final consideration is maintainability over time. As new records and search options are added, the team should keep a short change log, update the test cases, and review whether the current data structures still suit the workload. Small, well-documented changes are easier to verify than large rewrites. The project can therefore grow in stages: establish correct results first, improve the user experience next, and only then optimise measured bottlenecks or introduce advanced discovery techniques.

4.16 Trie and Autocomplete

A trie is a tree used to store strings by their character prefixes. Each node represents a prefix, and outgoing edges represent the next character. When several vocabulary words share a prefix, they share the same path. For example, “compute,” “computer,” and “computing” share the initial characters “comput.” A trie can therefore support prefix queries without checking every word independently.

Autocomplete begins when the user types a partial query. The frontend can request suggestions or use a local vocabulary supplied by the backend. The trie follows the prefix characters and then enumerates terminal words below the matching node. The number of suggestions should be limited to avoid overwhelming the user. Suggestions may be ordered by frequency, alphabetical order, or another clearly defined criterion.

Trie memory use depends on the number of distinct prefixes and the representation of child links. A map of children is flexible for a broad character set, while fixed arrays may be faster for a restricted alphabet but use more space. The application should handle an empty prefix, a prefix that has no matches, and input containing spaces or non-ASCII characters. Autocomplete is a query-assistance feature; it should not be confused with approximate spelling correction.

In practical use, the algorithm must be treated as one stage in a complete retrieval pipeline. The input should be normalised consistently, the output should be mapped back to paper identifiers, and duplicate candidates should be removed where the user expects one result per paper. Developers should document whether matching is case-sensitive, how positions are counted, and how empty input is handled. These small decisions affect both correctness and the way results are explained in the interface.

A useful verification method is to compare the implementation with a simple reference method on a small dataset whose answer can be checked manually. Include ordinary examples as well as boundary cases, then repeat the tests after any refactoring. Complexity analysis predicts how work grows, but measured performance also depends on data layout, allocation, text length, and the number of matches returned. This combination of correctness tests and measurement makes the feature easier to trust and maintain.

The system should also be assessed from the perspective of the person using it. A technically correct output is not enough if the user cannot understand why it appeared or what action to take next. Labels, examples, sensible defaults, and clear empty states reduce confusion. Where the system makes an approximate decision, such as suggesting a spelling correction or a related paper, the interface should communicate that uncertainty and let the user inspect the underlying record before relying on it.

Finally, the implementation should remain aligned with the evidence available in the project. Features that are implemented should be supported by a demonstration or test; features that are planned should be described as future work. This distinction is especially important in an academic report, where clear reasoning and honest limitations are stronger than unsupported claims. Keeping the documentation, source code, and demonstration consistent will make the project easier to evaluate and extend.

A final consideration is maintainability over time. As new records and search options are added, the team should keep a short change log, update the test cases, and review whether the current data structures still suit the workload. Small, well-documented changes are easier to verify than large rewrites. The project can therefore grow in stages: establish correct results first, improve the user experience next, and only then optimise measured bottlenecks or introduce advanced discovery techniques.

4.17 Backend Controllers and Services

The backend should organise endpoints around user-facing capabilities, such as search, paper details, similarity, citation exploration, and statistics. Controllers receive HTTP requests and return responses. Service classes coordinate access to the corpus, index structures, and algorithm classes. This division reduces the risk of placing complex algorithm logic directly inside request-handling methods.

A service should validate that a referenced paper exists before attempting similarity or citation analysis. Search services should apply filters consistently regardless of which matching algorithm generated the candidates. Shared validation and response mapping can reduce duplication. If a request fails, the backend should return an appropriate status and a concise explanation while recording technical details in logs for developers.

Unit tests can call algorithm and service methods directly. Controller tests can verify routing, parameter validation, status codes, and JSON structure. Integration tests can start the application with a known small corpus and exercise the complete request flow. This layered testing strategy helps identify whether an error is caused by the algorithm, application workflow, or HTTP configuration.

A clear separation of responsibilities also helps the team work in parallel. One member can improve the interface while another tests backend services, provided both agree on request fields and response structures. The contract should state required fields, optional fields, error cases, and the meaning of returned values. This avoids fragile assumptions in which one component expects a field that another component does not provide.

For maintenance, keep configuration outside business logic and avoid duplicating endpoint addresses or validation rules across multiple components. Use consistent naming, small methods, and logs that identify the failed operation without exposing unnecessary internal details. A setup guide should explain how to install dependencies, configure local ports, start each service, and diagnose common connection problems. These practices make the project easier to reproduce during assessment.

The system should also be assessed from the perspective of the person using it. A technically correct output is not enough if the user cannot understand why it appeared or what action to take next. Labels, examples, sensible defaults, and clear empty states reduce confusion. Where the system makes an approximate decision, such as suggesting a spelling correction or a related paper, the interface should communicate that uncertainty and let the user inspect the underlying record before relying on it.

Finally, the implementation should remain aligned with the evidence available in the project. Features that are implemented should be supported by a demonstration or test; features that are planned should be described as future work. This distinction is especially important in an academic report, where clear reasoning and honest limitations are stronger than unsupported claims. Keeping the documentation, source code, and demonstration consistent will make the project easier to evaluate and extend.

A final consideration is maintainability over time. As new records and search options are added, the team should keep a short change log, update the test cases, and review whether the current data structures still suit the workload. Small, well-documented changes are easier to verify than large rewrites. The project can therefore grow in stages: establish correct results first, improve the user experience next, and only then optimise measured bottlenecks or introduce advanced discovery techniques.

4.18 Frontend Components and State

A component-based frontend divides the page into smaller reusable pieces. A search bar handles query entry; a filter panel captures optional constraints; a results list renders returned records; a paper card displays summary metadata; and detail or graph components present more specialised information. Reuse keeps the interface consistent and makes individual features easier to update.

State management is important because several values change during a search session. The interface may need to track the query text, selected filters, current results, selected paper, active tab, loading state, and error state. When a request starts, loading should become true; after success or failure, it should become false. Old results should not be mistaken for the response to a new query, particularly if requests finish out of order.

Frontend testing should cover input validation, empty results, long titles, missing metadata, API errors, filter changes, and navigation between result and detail views. The visual design should remain readable at different window sizes. Clear labels and keyboard focus states improve accessibility. The frontend should display only the fields the backend actually supplies and should not fabricate publication details when a record is incomplete.

A clear separation of responsibilities also helps the team work in parallel. One member can improve the interface while another tests backend services, provided both agree on request fields and response structures. The contract should state required fields, optional fields, error cases, and the meaning of returned values. This avoids fragile assumptions in which one component expects a field that another component does not provide.

For maintenance, keep configuration outside business logic and avoid duplicating endpoint addresses or validation rules across multiple components. Use consistent naming, small methods, and logs that identify the failed operation without exposing unnecessary internal details. A setup guide should explain how to install dependencies, configure local ports, start each service, and diagnose common connection problems. These practices make the project easier to reproduce during assessment.

The system should also be assessed from the perspective of the person using it. A technically correct output is not enough if the user cannot understand why it appeared or what action to take next. Labels, examples, sensible defaults, and clear empty states reduce confusion. Where the system makes an approximate decision, such as suggesting a spelling correction or a related paper, the interface should communicate that uncertainty and let the user inspect the underlying record before relying on it.

Finally, the implementation should remain aligned with the evidence available in the project. Features that are implemented should be supported by a demonstration or test; features that are planned should be described as future work. This distinction is especially important in an academic report, where clear reasoning and honest limitations are stronger than unsupported claims. Keeping the documentation, source code, and demonstration consistent will make the project easier to evaluate and extend.

A final consideration is maintainability over time. As new records and search options are added, the team should keep a short change log, update the test cases, and review whether the current data structures still suit the workload. Small, well-documented changes are easier to verify than large rewrites. The project can therefore grow in stages: establish correct results first, improve the user experience next, and only then optimise measured bottlenecks or introduce advanced discovery techniques.

4.19 REST API Integration and CORS

The frontend and backend often run on separate local development ports. Browsers enforce cross-origin restrictions, so the backend must allow requests from the intended frontend origin. Spring Boot can configure CORS centrally or at selected controllers. During deployment, the allowed origin list should be narrowed to the actual application domains rather than allowing every origin without justification.

API integration should centralise the backend base URL so that components do not hard-code different addresses. Request helpers can standardise query encoding, headers, JSON parsing, and error handling. A network error differs from an HTTP error: the server may be unreachable, or it may respond with a status indicating that the request was invalid or unsuccessful. The interface should distinguish these cases where useful.

If the API returns a structured error, the frontend can display the message without exposing internal stack traces. Timeouts and cancellation may be useful for long-running operations. Large similarity requests should be bounded with limits or pagination. Consistent API behaviour is essential for a dependable full-stack application and makes future deployment easier.

A clear separation of responsibilities also helps the team work in parallel. One member can improve the interface while another tests backend services, provided both agree on request fields and response structures. The contract should state required fields, optional fields, error cases, and the meaning of returned values. This avoids fragile assumptions in which one component expects a field that another component does not provide.

For maintenance, keep configuration outside business logic and avoid duplicating endpoint addresses or validation rules across multiple components. Use consistent naming, small methods, and logs that identify the failed operation without exposing unnecessary internal details. A setup guide should explain how to install dependencies, configure local ports, start each service, and diagnose common connection problems. These practices make the project easier to reproduce during assessment.

The system should also be assessed from the perspective of the person using it. A technically correct output is not enough if the user cannot understand why it appeared or what action to take next. Labels, examples, sensible defaults, and clear empty states reduce confusion. Where the system makes an approximate decision, such as suggesting a spelling correction or a related paper, the interface should communicate that uncertainty and let the user inspect the underlying record before relying on it.

Finally, the implementation should remain aligned with the evidence available in the project. Features that are implemented should be supported by a demonstration or test; features that are planned should be described as future work. This distinction is especially important in an academic report, where clear reasoning and honest limitations are stronger than unsupported claims. Keeping the documentation, source code, and demonstration consistent will make the project easier to evaluate and extend.

A final consideration is maintainability over time. As new records and search options are added, the team should keep a short change log, update the test cases, and review whether the current data structures still suit the workload. Small, well-documented changes are easier to verify than large rewrites. The project can therefore grow in stages: establish correct results first, improve the user experience next, and only then optimise measured bottlenecks or introduce advanced discovery techniques.

4.20 Search Filtering and Ranking

Filtering narrows an existing candidate set according to selected criteria. Year filtering keeps records within the requested publication period, while category filtering keeps records belonging to a selected research domain. Filters should be applied using a consistent definition of missing values and should not unexpectedly remove valid records because metadata formatting differs. If users can combine filters, the application must define whether conditions are combined with logical AND or OR.

Ranking orders the results so that likely useful records appear first. The simplest implementation may preserve corpus order or prioritise exact title matches. More advanced ranking could use term frequency, inverse document frequency, field weights, citation signals, or semantic representations. The current report only treats these as potential strategies unless the implementation explicitly contains them. A ranking score should be explainable enough for users and testable against representative queries.

An evaluation set can contain queries with expected relevant records. The team can inspect the top results and record whether they are relevant, duplicated, or misleading. Precision at a selected cutoff and recall over known relevant items are possible metrics when ground truth is available. For a small educational prototype, a carefully documented manual evaluation may be more honest than presenting numerical metrics without a labelled test set.

The practical value of this section is that it connects the project design to decisions that can be verified during implementation. Each field, algorithm, or workflow should have a defined purpose, and its output should be understandable to the next module that uses it. Explicit assumptions reduce ambiguity for both developers and reviewers. When an assumption changes, related tests and documentation should be updated together.

A good next step is to validate this part of the system with representative examples, including missing data and unexpected input. The team should note what works, what fails, and what remains unimplemented. This evidence-based approach keeps the final report aligned with the source code and gives a practical starting point for future improvements.

The system should also be assessed from the perspective of the person using it. A technically correct output is not enough if the user cannot understand why it appeared or what action to take next. Labels, examples, sensible defaults, and clear empty states reduce confusion. Where the system makes an approximate decision, such as suggesting a spelling correction or a related paper, the interface should communicate that uncertainty and let the user inspect the underlying record before relying on it.

Finally, the implementation should remain aligned with the evidence available in the project. Features that are implemented should be supported by a demonstration or test; features that are planned should be described as future work. This distinction is especially important in an academic report, where clear reasoning and honest limitations are stronger than unsupported claims. Keeping the documentation, source code, and demonstration consistent will make the project easier to evaluate and extend.

A final consideration is maintainability over time. As new records and search options are added, the team should keep a short change log, update the test cases, and review whether the current data structures still suit the workload. Small, well-documented changes are easier to verify than large rewrites. The project can therefore grow in stages: establish correct results first, improve the user experience next, and only then optimise measured bottlenecks or introduce advanced discovery techniques.

CHAPTER 5 — TESTING AND RESULTS

5.1 Testing Strategy

Testing verifies that the application behaves as intended and helps prevent changes from breaking existing functions. Unit tests should cover individual algorithms with small known inputs. Service tests should verify how candidate records, filters, and algorithm outputs are combined. API tests should confirm routing, validation, response formats, and error handling. End-to-end tests should exercise the main user journeys through the frontend and backend together.

A useful test plan links every functional requirement to at least one test case. The test record should include an identifier, preconditions, input, expected output, observed output, and pass/fail status. If a test cannot be run, it should be marked as not executed rather than reported as passed. Screenshots can provide evidence of interface behaviour, but they do not replace tests of algorithm correctness or data integrity.

Testing should use a small, controlled dataset where expected results can be calculated manually. For example, a few short text records can be used to verify KMP positions, multi-pattern matches, edit distances, and citation traversal order. Once those tests pass, a larger corpus can be used to check integration and responsiveness. This progression makes defects easier to isolate.

Test evidence should be reproducible. Record the dataset version, input query, expected outcome, actual outcome, and whether the test passed. For timing tests, record the machine and runtime configuration and repeat each measurement rather than relying on a single run. A result table should distinguish observations from interpretation, and should never include invented numbers simply to make the project appear faster or more accurate.

When a test fails, reduce the problem to the smallest input that still reproduces the error. This often reveals whether the cause is an algorithm boundary case, inconsistent data, an API contract mismatch, or a frontend state problem. Fix the underlying cause and rerun both the failing test and nearby regression tests. Keeping a small regression suite prevents previously corrected issues from returning during later development.

The system should also be assessed from the perspective of the person using it. A technically correct output is not enough if the user cannot understand why it appeared or what action to take next. Labels, examples, sensible defaults, and clear empty states reduce confusion. Where the system makes an approximate decision, such as suggesting a spelling correction or a related paper, the interface should communicate that uncertainty and let the user inspect the underlying record before relying on it.

Finally, the implementation should remain aligned with the evidence available in the project. Features that are implemented should be supported by a demonstration or test; features that are planned should be described as future work. This distinction is especially important in an academic report, where clear reasoning and honest limitations are stronger than unsupported claims. Keeping the documentation, source code, and demonstration consistent will make the project easier to evaluate and extend.

A final consideration is maintainability over time. As new records and search options are added, the team should keep a short change log, update the test cases, and review whether the current data structures still suit the workload. Small, well-documented changes are easier to verify than large rewrites. The project can therefore grow in stages: establish correct results first, improve the user experience next, and only then optimise measured bottlenecks or introduce advanced discovery techniques.

5.2 KMP and Exact Search Test Cases

KMP tests should verify successful and unsuccessful exact matches. A pattern located at the beginning of a text checks the initial comparison path. A pattern at the end checks that the scan continues correctly to the final characters. Multiple occurrences check whether the implementation reports all matches or intentionally stops after the first. A pattern longer than the text should return no match without causing an indexing error.

Repeated-character patterns are particularly valuable because they exercise the LPS logic. Text such as “aaaaab” and pattern “aaab” can reveal incorrect fallback behaviour. Empty text and empty pattern handling must be specified. If the application normalises case, tests should confirm that the same normalisation is applied to both query and document text.

At corpus level, indexed search should be compared with a straightforward reference scan on a sample collection. Every record containing the exact pattern under the defined matching rules should be present in the result set. If snippets are shown, the snippet should correspond to the matched record. The system should also avoid duplicate result cards when a term appears many times in one paper.

In practical use, the algorithm must be treated as one stage in a complete retrieval pipeline. The input should be normalised consistently, the output should be mapped back to paper identifiers, and duplicate candidates should be removed where the user expects one result per paper. Developers should document whether matching is case-sensitive, how positions are counted, and how empty input is handled. These small decisions affect both correctness and the way results are explained in the interface.

A useful verification method is to compare the implementation with a simple reference method on a small dataset whose answer can be checked manually. Include ordinary examples as well as boundary cases, then repeat the tests after any refactoring. Complexity analysis predicts how work grows, but measured performance also depends on data layout, allocation, text length, and the number of matches returned. This combination of correctness tests and measurement makes the feature easier to trust and maintain.

The system should also be assessed from the perspective of the person using it. A technically correct output is not enough if the user cannot understand why it appeared or what action to take next. Labels, examples, sensible defaults, and clear empty states reduce confusion. Where the system makes an approximate decision, such as suggesting a spelling correction or a related paper, the interface should communicate that uncertainty and let the user inspect the underlying record before relying on it.

Finally, the implementation should remain aligned with the evidence available in the project. Features that are implemented should be supported by a demonstration or test; features that are planned should be described as future work. This distinction is especially important in an academic report, where clear reasoning and honest limitations are stronger than unsupported claims. Keeping the documentation, source code, and demonstration consistent will make the project easier to evaluate and extend.

A final consideration is maintainability over time. As new records and search options are added, the team should keep a short change log, update the test cases, and review whether the current data structures still suit the workload. Small, well-documented changes are easier to verify than large rewrites. The project can therefore grow in stages: establish correct results first, improve the user experience next, and only then optimise measured bottlenecks or introduce advanced discovery techniques.

5.3 Aho–Corasick Test Cases

Aho–Corasick tests should begin with a small pattern set whose matches can be listed by hand. Include patterns with shared prefixes, patterns where one pattern is a suffix of another, and overlapping occurrences. The test should verify that failure links lead to valid states and that all intended patterns are reported without repeating the same occurrence incorrectly.

A text containing none of the patterns should return an empty match set. A text containing the same pattern several times should follow the documented output policy. If the search uses a normalised vocabulary, test case variations and punctuation. If duplicate patterns are supplied, the implementation should either deduplicate them during preprocessing or document how duplicates appear in the output.

At application level, multi-keyword search should return paper identifiers that contain at least one of the requested terms, or all of them, according to the selected search semantics. This distinction should be explicit in the interface. Test results should record the actual output from the running implementation rather than assuming that the algorithm description alone proves the feature works.

In practical use, the algorithm must be treated as one stage in a complete retrieval pipeline. The input should be normalised consistently, the output should be mapped back to paper identifiers, and duplicate candidates should be removed where the user expects one result per paper. Developers should document whether matching is case-sensitive, how positions are counted, and how empty input is handled. These small decisions affect both correctness and the way results are explained in the interface.

A useful verification method is to compare the implementation with a simple reference method on a small dataset whose answer can be checked manually. Include ordinary examples as well as boundary cases, then repeat the tests after any refactoring. Complexity analysis predicts how work grows, but measured performance also depends on data layout, allocation, text length, and the number of matches returned. This combination of correctness tests and measurement makes the feature easier to trust and maintain.

The system should also be assessed from the perspective of the person using it. A technically correct output is not enough if the user cannot understand why it appeared or what action to take next. Labels, examples, sensible defaults, and clear empty states reduce confusion. Where the system makes an approximate decision, such as suggesting a spelling correction or a related paper, the interface should communicate that uncertainty and let the user inspect the underlying record before relying on it.

Finally, the implementation should remain aligned with the evidence available in the project. Features that are implemented should be supported by a demonstration or test; features that are planned should be described as future work. This distinction is especially important in an academic report, where clear reasoning and honest limitations are stronger than unsupported claims. Keeping the documentation, source code, and demonstration consistent will make the project easier to evaluate and extend.

A final consideration is maintainability over time. As new records and search options are added, the team should keep a short change log, update the test cases, and review whether the current data structures still suit the workload. Small, well-documented changes are easier to verify than large rewrites. The project can therefore grow in stages: establish correct results first, improve the user experience next, and only then optimise measured bottlenecks or introduce advanced discovery techniques.

5.4 Edit Distance Test Cases

Edit-distance tests should include identical strings, where the distance is zero; one insertion; one deletion; one substitution; and strings that differ in several positions. The implementation should correctly initialise the first row and column of the dynamic-programming table. Empty strings are important edge cases because their distance is equal to the length of the non-empty string.

Suggestion tests should cover both a plausible typo and an unrelated query. For example, a single missing character in a known vocabulary term should usually produce a small distance, while a very different word should not be suggested if it exceeds the chosen threshold. Short queries should be tested separately because an edit threshold that is appropriate for long words may be too permissive for short ones.

The quality of suggestions should be reviewed for false positives. A low edit distance can still point to the wrong technical term. The interface should let users choose whether to search the suggested term and should preserve the original query. Evaluation should record both useful corrections and misleading suggestions to guide threshold tuning.

In practical use, the algorithm must be treated as one stage in a complete retrieval pipeline. The input should be normalised consistently, the output should be mapped back to paper identifiers, and duplicate candidates should be removed where the user expects one result per paper. Developers should document whether matching is case-sensitive, how positions are counted, and how empty input is handled. These small decisions affect both correctness and the way results are explained in the interface.

A useful verification method is to compare the implementation with a simple reference method on a small dataset whose answer can be checked manually. Include ordinary examples as well as boundary cases, then repeat the tests after any refactoring. Complexity analysis predicts how work grows, but measured performance also depends on data layout, allocation, text length, and the number of matches returned. This combination of correctness tests and measurement makes the feature easier to trust and maintain.

The system should also be assessed from the perspective of the person using it. A technically correct output is not enough if the user cannot understand why it appeared or what action to take next. Labels, examples, sensible defaults, and clear empty states reduce confusion. Where the system makes an approximate decision, such as suggesting a spelling correction or a related paper, the interface should communicate that uncertainty and let the user inspect the underlying record before relying on it.

Finally, the implementation should remain aligned with the evidence available in the project. Features that are implemented should be supported by a demonstration or test; features that are planned should be described as future work. This distinction is especially important in an academic report, where clear reasoning and honest limitations are stronger than unsupported claims. Keeping the documentation, source code, and demonstration consistent will make the project easier to evaluate and extend.

A final consideration is maintainability over time. As new records and search options are added, the team should keep a short change log, update the test cases, and review whether the current data structures still suit the workload. Small, well-documented changes are easier to verify than large rewrites. The project can therefore grow in stages: establish correct results first, improve the user experience next, and only then optimise measured bottlenecks or introduce advanced discovery techniques.

5.5 Similarity and Citation Tests

Similarity tests should use a few papers with deliberately overlapping and non-overlapping text. The system should identify the known overlap, exclude the selected paper from its own recommendation list, and avoid duplicate candidates. If similarity depends on suffix-array positions, the test should verify that offsets map back to the correct source paper and that document separators are not treated as ordinary content.

Citation graph tests should include a single edge, a chain, branching paths, disconnected nodes, and a cycle. BFS should visit reachable nodes in increasing hop distance, while DFS should reach all reachable nodes according to its traversal rules. Each node should be processed at most once per traversal. Directed edges must be followed in the intended direction, and unresolved external citations should be handled consistently.

The interface should make graph results understandable. It can show the selected paper, its directly connected records, and the meaning of each edge. A graph traversal test should check the data returned by the backend as well as what the user sees. If a visual graph is planned but not present in the supplied build, it should be documented as a future enhancement rather than a completed result.

The practical value of this section is that it connects the project design to decisions that can be verified during implementation. Each field, algorithm, or workflow should have a defined purpose, and its output should be understandable to the next module that uses it. Explicit assumptions reduce ambiguity for both developers and reviewers. When an assumption changes, related tests and documentation should be updated together.

A good next step is to validate this part of the system with representative examples, including missing data and unexpected input. The team should note what works, what fails, and what remains unimplemented. This evidence-based approach keeps the final report aligned with the source code and gives a practical starting point for future improvements.

The system should also be assessed from the perspective of the person using it. A technically correct output is not enough if the user cannot understand why it appeared or what action to take next. Labels, examples, sensible defaults, and clear empty states reduce confusion. Where the system makes an approximate decision, such as suggesting a spelling correction or a related paper, the interface should communicate that uncertainty and let the user inspect the underlying record before relying on it.

Finally, the implementation should remain aligned with the evidence available in the project. Features that are implemented should be supported by a demonstration or test; features that are planned should be described as future work. This distinction is especially important in an academic report, where clear reasoning and honest limitations are stronger than unsupported claims. Keeping the documentation, source code, and demonstration consistent will make the project easier to evaluate and extend.

A final consideration is maintainability over time. As new records and search options are added, the team should keep a short change log, update the test cases, and review whether the current data structures still suit the workload. Small, well-documented changes are easier to verify than large rewrites. The project can therefore grow in stages: establish correct results first, improve the user experience next, and only then optimise measured bottlenecks or introduce advanced discovery techniques.

5.6 Filtering, Statistics and Interface Tests

Filtering tests should confirm that a selected year range includes only records within the range and that a category filter returns only records assigned to that category. Combined filters should follow a documented logical rule. Clearing filters should restore the broader result set. Missing or malformed metadata should not crash the page and should be represented consistently.

Statistics tests should compare displayed counts with the loaded corpus. The total paper count should reflect the actual number of records loaded, not a hard-coded expected number if loading failed. Category and year totals should add up according to the counting rules. Author counts need a defined policy for records with multiple authors or inconsistent author names.

Interface tests should include slow responses, server errors, empty results, long titles, missing fields, and repeated submissions. Loading indicators should disappear after success or failure. Search controls should remain usable with keyboard navigation. These tests help ensure that correct algorithmic output is presented in a way that users can understand and trust.

A clear separation of responsibilities also helps the team work in parallel. One member can improve the interface while another tests backend services, provided both agree on request fields and response structures. The contract should state required fields, optional fields, error cases, and the meaning of returned values. This avoids fragile assumptions in which one component expects a field that another component does not provide.

For maintenance, keep configuration outside business logic and avoid duplicating endpoint addresses or validation rules across multiple components. Use consistent naming, small methods, and logs that identify the failed operation without exposing unnecessary internal details. A setup guide should explain how to install dependencies, configure local ports, start each service, and diagnose common connection problems. These practices make the project easier to reproduce during assessment.

The system should also be assessed from the perspective of the person using it. A technically correct output is not enough if the user cannot understand why it appeared or what action to take next. Labels, examples, sensible defaults, and clear empty states reduce confusion. Where the system makes an approximate decision, such as suggesting a spelling correction or a related paper, the interface should communicate that uncertainty and let the user inspect the underlying record before relying on it.

Finally, the implementation should remain aligned with the evidence available in the project. Features that are implemented should be supported by a demonstration or test; features that are planned should be described as future work. This distinction is especially important in an academic report, where clear reasoning and honest limitations are stronger than unsupported claims. Keeping the documentation, source code, and demonstration consistent will make the project easier to evaluate and extend.

A final consideration is maintainability over time. As new records and search options are added, the team should keep a short change log, update the test cases, and review whether the current data structures still suit the workload. Small, well-documented changes are easier to verify than large rewrites. The project can therefore grow in stages: establish correct results first, improve the user experience next, and only then optimise measured bottlenecks or introduce advanced discovery techniques.

5.7 Results and Interpretation

The project report describes an integrated set of functions: exact keyword search, multiple-keyword search, typo-tolerant search, textual-pattern similarity, citation traversal, indexed candidate retrieval, filtering, and collection statistics. These features demonstrate how different algorithms can be assigned to separate research-discovery tasks. Their value lies in combining them within one workflow rather than relying on one search technique for every need.

KMP and Aho–Corasick address exact matching at different scales: one pattern versus many patterns. Edit distance supports approximate spelling correction. Suffix-array and LCP methods identify repeated text fragments. BFS and DFS reveal graph reachability and paths. Hashing and inverted indexing help locate candidate records. These roles are distinct, so the results should be interpreted according to the method used for each operation.

No numerical latency, precision, recall, or user-satisfaction figures are asserted here because the supplied report does not provide a reproducible benchmark table. A stronger evaluation would specify hardware, corpus size, average document length, query set, number of repetitions, and timing method. It would also compare indexed search with a baseline and assess recommendation relevance with a labelled set or expert review.

Test evidence should be reproducible. Record the dataset version, input query, expected outcome, actual outcome, and whether the test passed. For timing tests, record the machine and runtime configuration and repeat each measurement rather than relying on a single run. A result table should distinguish observations from interpretation, and should never include invented numbers simply to make the project appear faster or more accurate.

When a test fails, reduce the problem to the smallest input that still reproduces the error. This often reveals whether the cause is an algorithm boundary case, inconsistent data, an API contract mismatch, or a frontend state problem. Fix the underlying cause and rerun both the failing test and nearby regression tests. Keeping a small regression suite prevents previously corrected issues from returning during later development.

The system should also be assessed from the perspective of the person using it. A technically correct output is not enough if the user cannot understand why it appeared or what action to take next. Labels, examples, sensible defaults, and clear empty states reduce confusion. Where the system makes an approximate decision, such as suggesting a spelling correction or a related paper, the interface should communicate that uncertainty and let the user inspect the underlying record before relying on it.

Finally, the implementation should remain aligned with the evidence available in the project. Features that are implemented should be supported by a demonstration or test; features that are planned should be described as future work. This distinction is especially important in an academic report, where clear reasoning and honest limitations are stronger than unsupported claims. Keeping the documentation, source code, and demonstration consistent will make the project easier to evaluate and extend.

A final consideration is maintainability over time. As new records and search options are added, the team should keep a short change log, update the test cases, and review whether the current data structures still suit the workload. Small, well-documented changes are easier to verify than large rewrites. The project can therefore grow in stages: establish correct results first, improve the user experience next, and only then optimise measured bottlenecks or introduce advanced discovery techniques.

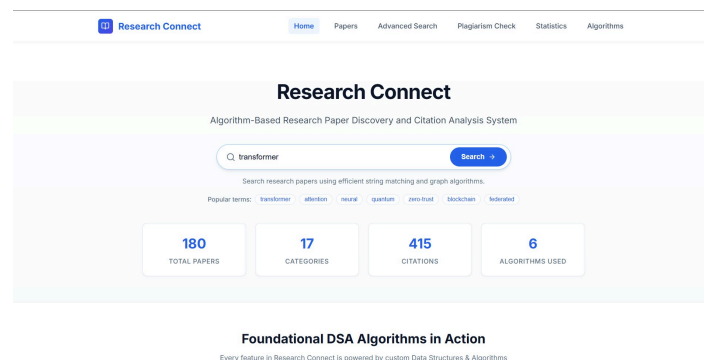


Figure 5.1: Research Connect home page

5.8 Performance Analysis and Complexity

Algorithmic complexity helps estimate how processing cost grows as input size increases. KMP runs in $O(N + M)$ time for text length N and pattern length M . Aho–Corasick is commonly described as $O(P + N + Z)$, where P is the total pattern length, N is the text length, and Z is the number of reported matches. Levenshtein dynamic programming takes $O(MN)$ time for two strings of lengths M and N , with a full table requiring $O(MN)$ space.

BFS and DFS take $O(V + E)$ time with adjacency lists, where V is the number of graph vertices and E is the number of edges. HashMap operations are expected to be constant-time on average, while an inverted index reduces candidate generation to the postings for the query terms plus the cost of processing those candidates. Suffix-array construction complexity depends on the selected algorithm; the implementation should state the actual approach rather than assume an optimal bound.

End-to-end latency includes more than algorithm runtime. Request handling, serialisation, network transfer, sorting, filtering, and browser rendering all contribute. Performance experiments should measure these stages separately where possible. For small datasets, network or UI overhead may dominate; for larger datasets, index construction and text processing may become more significant. Optimisation should follow measurement rather than guesswork.

Test evidence should be reproducible. Record the dataset version, input query, expected outcome, actual outcome, and whether the test passed. For timing tests, record the machine and runtime configuration and repeat each measurement rather than relying on a single run. A result table should distinguish observations from interpretation, and should never include invented numbers simply to make the project appear faster or more accurate.

When a test fails, reduce the problem to the smallest input that still reproduces the error. This often reveals whether the cause is an algorithm boundary case, inconsistent data, an API contract mismatch, or a frontend state problem. Fix the underlying cause and rerun both the failing test and nearby regression tests. Keeping a small regression suite prevents previously corrected issues from returning during later development.

The system should also be assessed from the perspective of the person using it. A technically correct output is not enough if the user cannot understand why it appeared or what action to take next. Labels, examples, sensible defaults, and clear empty states reduce confusion. Where the system makes an approximate decision, such as suggesting a spelling correction or a related paper, the interface should communicate that uncertainty and let the user inspect the underlying record before relying on it.

Finally, the implementation should remain aligned with the evidence available in the project. Features that are implemented should be supported by a demonstration or test; features that are planned should be described as future work. This distinction is especially important in an academic report, where clear reasoning and honest limitations are stronger than unsupported claims. Keeping the documentation, source code, and demonstration consistent will make the project easier to evaluate and extend.

A final consideration is maintainability over time. As new records and search options are added, the team should keep a short change log, update the test cases, and review whether the current data structures still suit the workload. Small, well-documented changes are easier to verify than large rewrites. The project can therefore grow in stages: establish correct results first, improve the user experience next, and only then optimise measured bottlenecks or introduce advanced discovery techniques.

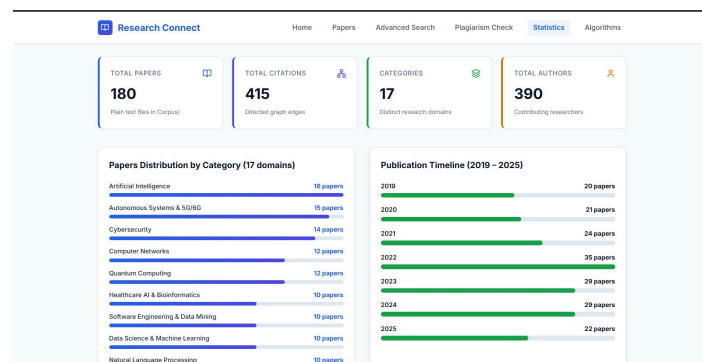


Figure 5.2: Research-paper collection statistics

CHAPTER 6 — CHALLENGES AND LIMITATIONS

6.1 Development Challenges

One challenge is combining algorithms that expect different forms of input. KMP and Aho–Corasick operate on strings, edit distance compares words, suffix arrays require careful text-position mapping, and graph algorithms operate on identifiers and edges. The application must convert paper records into the appropriate representations while preserving a link back to the source record. A defect in normalisation or identifier mapping can affect several features at once.

A second challenge is maintaining consistent data. Titles, authors, years, categories, and citation relationships may be missing or inconsistent. Duplicate records can create repeated results, and references to unknown IDs can create broken graph links. A controlled loading and validation process reduces these problems, but a larger real-world corpus would require ongoing data-quality checks.

A third challenge is integration. The frontend and backend may run on different ports, and an incorrect API URL or CORS policy can prevent requests from succeeding. Response fields must remain consistent between Java objects and React components. Modular design, clear API contracts, logging, and incremental testing help isolate such problems.

The practical value of this section is that it connects the project design to decisions that can be verified during implementation. Each field, algorithm, or workflow should have a defined purpose, and its output should be understandable to the next module that uses it. Explicit assumptions reduce ambiguity for both developers and reviewers. When an assumption changes, related tests and documentation should be updated together.

A good next step is to validate this part of the system with representative examples, including missing data and unexpected input. The team should note what works, what fails, and what remains unimplemented. This evidence-based approach keeps the final report aligned with the source code and gives a practical starting point for future improvements.

The system should also be assessed from the perspective of the person using it. A technically correct output is not enough if the user cannot understand why it appeared or what action to take next. Labels, examples, sensible defaults, and clear empty states reduce confusion. Where the system makes an approximate decision, such as suggesting a spelling correction or a related paper, the interface should communicate that uncertainty and let the user inspect the underlying record before relying on it.

Finally, the implementation should remain aligned with the evidence available in the project. Features that are implemented should be supported by a demonstration or test; features that are planned should be described as future work. This distinction is especially important in an academic report, where clear reasoning and honest limitations are stronger than unsupported claims. Keeping the documentation, source code, and demonstration consistent will make the project easier to evaluate and extend.

A final consideration is maintainability over time. As new records and search options are added, the team should keep a short change log, update the test cases, and review whether the current data structures still suit the workload. Small, well-documented changes are easier to verify than large rewrites. The project can therefore grow in stages: establish correct results first, improve the user experience next, and only then optimise measured bottlenecks or introduce advanced discovery techniques.

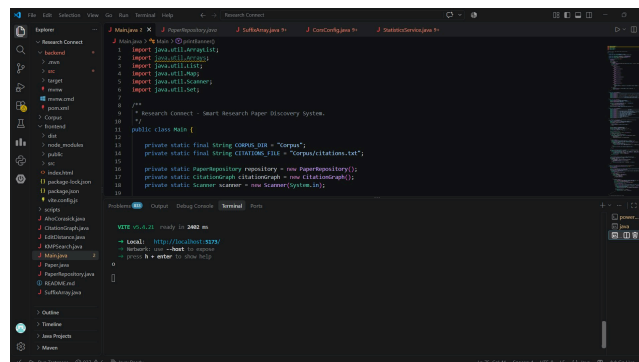


Figure 5.3: Development implementation screenshot

6.2 Solutions Applied

The project uses specialised algorithms for specialised tasks instead of relying on one universal search method. KMP supports exact matching, Aho–Corasick supports multi-pattern matching, edit distance supports spelling-tolerant suggestions, and graph traversal supports citation exploration. Hashing and inverted indexing help candidate retrieval. This division makes the purpose of each technique easier to explain and test.

The layered architecture separates presentation, request handling, algorithms, and data. React components handle display and state; Spring Boot controllers receive requests; services coordinate operations; and algorithm classes perform focused computations. This structure reduces coupling and allows unit tests to target algorithm logic independently of the browser.

Where the prototype has limitations, the correct solution is to document them and define measurable improvements. For example, a larger corpus requires index update strategies and performance testing; semantic discovery requires more than exact text overlap; and recommendation quality requires a relevance evaluation set. These are development opportunities rather than reasons to overstate current functionality.

The practical value of this section is that it connects the project design to decisions that can be verified during implementation. Each field, algorithm, or workflow should have a defined purpose, and its output should be understandable to the next module that uses it. Explicit assumptions reduce ambiguity for both developers and reviewers. When an assumption changes, related tests and documentation should be updated together.

A good next step is to validate this part of the system with representative examples, including missing data and unexpected input. The team should note what works, what fails, and what remains unimplemented. This evidence-based approach keeps the final report aligned with the source code and gives a practical starting point for future improvements.

The system should also be assessed from the perspective of the person using it. A technically correct output is not enough if the user cannot understand why it appeared or what action to take next. Labels, examples, sensible defaults, and clear empty states reduce confusion. Where the system makes an approximate decision, such as suggesting a spelling correction or a related paper, the interface should communicate that uncertainty and let the user inspect the underlying record before relying on it.

Finally, the implementation should remain aligned with the evidence available in the project. Features that are implemented should be supported by a demonstration or test; features that are planned should be described as future work. This distinction is especially important in an academic report, where clear reasoning and honest limitations are stronger than unsupported claims. Keeping the documentation, source code, and demonstration consistent will make the project easier to evaluate and extend.

A final consideration is maintainability over time. As new records and search options are added, the team should keep a short change log, update the test cases, and review whether the current data structures still suit the workload. Small, well-documented changes are easier to verify than large rewrites. The project can therefore grow in stages: establish correct results first, improve the user experience next, and only then optimise measured bottlenecks or introduce advanced discovery techniques.

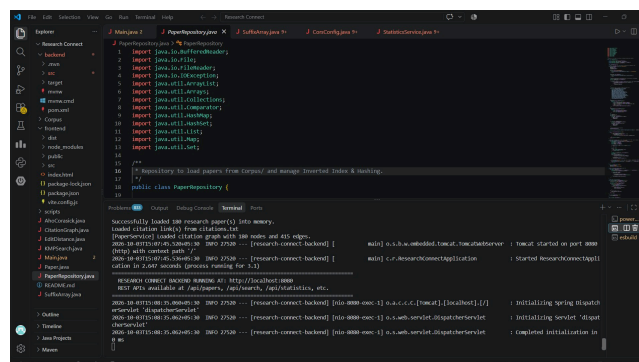


Figure 5.4: Development and execution screenshot